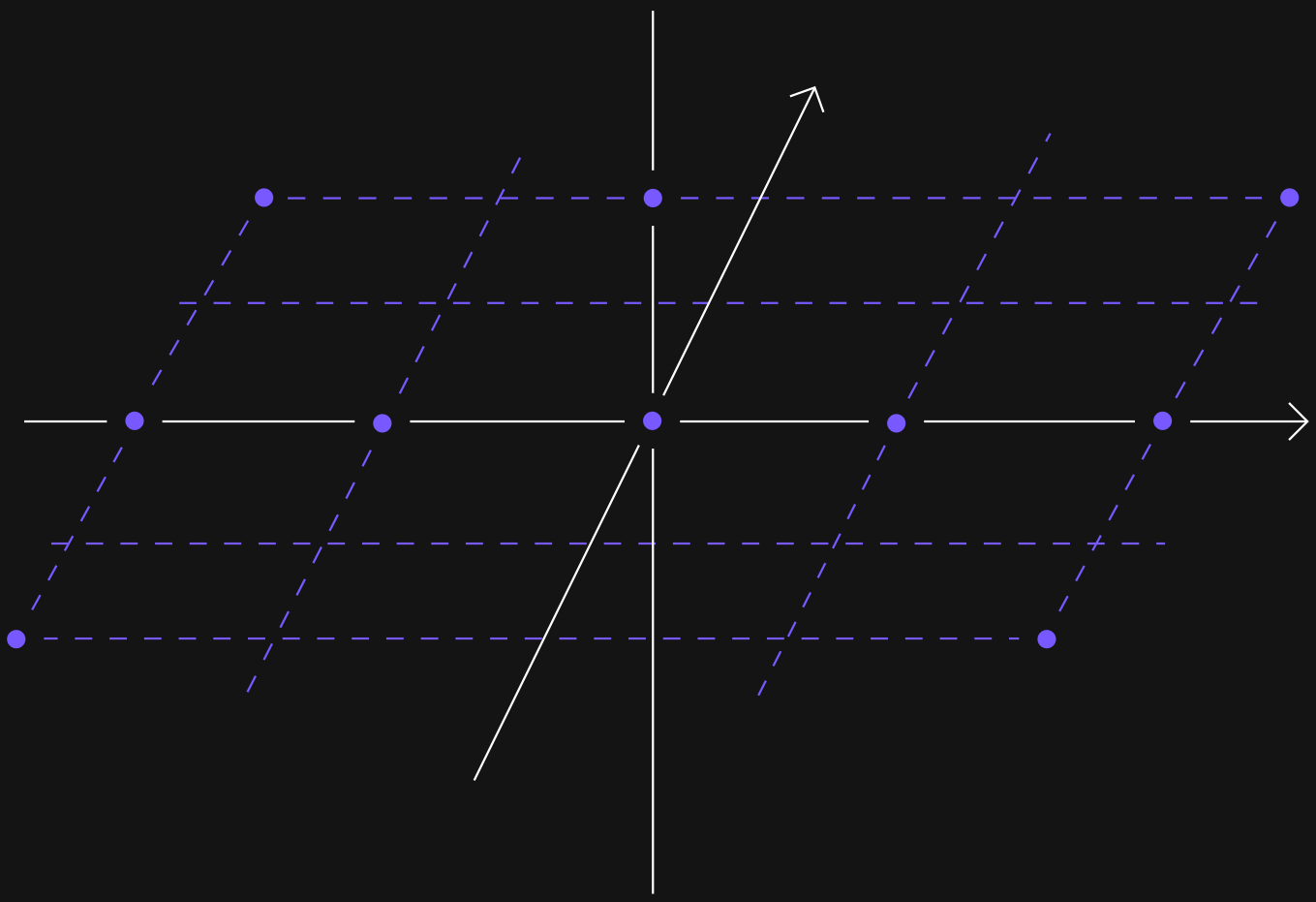
Машинное обучение

Лонгрид

Неделя 8

Случайный лес. Градиентный бустинг

ЧАСТЬ 1



Содержание

1. Случайный лес 4

1.1. Зачем усреднять деревья 4

1.2. Два источника случайности 5

1.3. Как обучается случайный лес 7

1.4. Как случайность снижает корреляцию 9

1.5. Считаем лес вручную 12

1.6. Гиперпараметры 15

1.7. Важность признаков 20

1.8. Extra Trees 21

1.9. Сильные и слабые стороны случайного леса 22

2. Градиентный бустинг 24

2.1. Идея бустинга 24

2.2. Считаем бустинг вручную на задаче регрессии 27

2.3. Произвольная функция потерь: от остатков к антиградиенту 33

2.4. Почему нельзя просто прибавить антиградиент 35

2.5. Алгоритм градиентного бустинга для любой базовой модели и произвольной функции потерь 38

2.6. Дерево изнутри: структура и значения листьев 39

2.7. Какое число положить в лист 42

2.8. Алгоритм градиентного бустинга над деревьями для произвольной функции потерь 44

2.9. Значение листа: ньютоновский шаг 46

2.10. Градиентный бустинг для классификации 49

2.11. Бустинг в sklearn 53

2.12. Чем управляют в бустинге 55

2.13. Сравнение случайного леса и градиентного бустинга 58

Если кратко 62

Случайный лес. Градиентный бустинг

Контекст

Неделя 7 показала, из чего складывается ошибка модели (смещение, разброс и неустранимый шум), и дала первый способ ею управлять. Параллельные ансамбли вроде бэггинга обучают много моделей на бутстрэп-выборках, усредняют их ответы и за счёт этого снижают разброс. Эта неделя доводит линию до конца и разбирает **случайный лес** — самый популярный параллельный ансамбль. Затем идея ансамблирования разворачивается на 180 градусов и превращается в **градиентный бустинг**, где модели не усредняются, а по очереди исправляют ошибки друг друга.

Применение

Случайный лес и градиентный бустинг остаются двумя главными рабочими инструментами для табличных данных. Лес — быстрый и надёжный baseline, он почти не требует настройки, и испортить его трудно. Бустинг требует аккуратности, зато остаётся одним из сильнейших семейств моделей для табличных данных и на большинстве задач превосходит универсальные baseline-модели. Именно на нём построены XGBoost, LightGBM и CatBoost, которые выигрывают соревнования и работают в проде в скоринге, рекомендациях, антифроде и прогнозировании спроса.

Содержание

Лонгрид состоит из двух частей. В части 1 разбирается случайный лес: два источника случайности, их влияние на корреляцию деревьев, гиперпараметры и ручной счёт на трёх деревьях. В части 2 разбирается градиентный бустинг: от ручного счёта на пяти квартирах до общего алгоритма для произвольной функции потерь и его версии для классификации.

Краткий план

→ Случайный лес
→ Градиентный бустинг
→ Если кратко

1. Случайный лес

- $a(x)$ — ансамбль,
- $b_k(x)$ — отдельная базовая модель,
- K — число базовых моделей,
- в обучающей выборке N объектов и p признаков,
- i — индекс объекта,
- k — индекс базовой модели,
- m — число признаков, доступных алгоритму на одном сплите.

1.1. ЗАЧЕМ УСРЕДНЯТЬ ДЕРЕВЬЯ

Глубокое неограниченное дерево — модель с **низким смещением (bias)** и **очень высоким разбросом (variance)**. Оно выразительно и способно приблизить сложную зависимость, поэтому его смещение мало. Отдельно от этого оно почти идеально подстраивается под обучающую выборку, и отсюда высокий разброс.

Но стоит поменять в данных одно наблюдение, и структура дерева может перестроиться целиком, а предсказания измениться до неузнаваемости. Такую модель и имеет смысл усреднять.

Разброс видно на одномерном примере (см. рисунок 1). Пять глубоких деревьев, обученных на разных бутстрэп-выборках, дают заметно разные ступенчатые кривые: на одних и тех же данных модель меняется вместе с выборкой. Среднее ста таких деревьев идёт рядом с истинной зависимостью, и это ровно тот эффект, ради которого деревья усредняют.

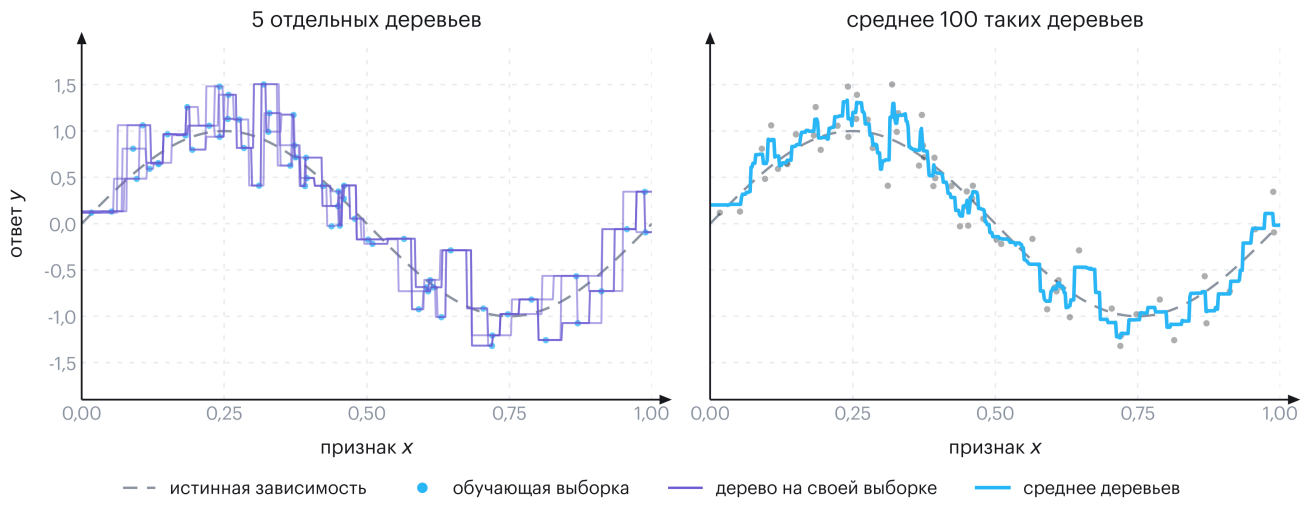


Рисунок 1. Отдельные деревья скачут от выборки к выборке, их среднее держится рядом с истинной зависимостью

Бэггинг именно это и делает: K деревьев обучаются на K бутстрэп-выборках, а ответом ансамбля служит среднее их прогнозов (в классификации — голосование большинством). Смещение при этом остаётся примерно тем же, что у одного дерева, а разброс падает.

Ключевой вывод недели 7 состоял в том, что величина падения определяется не столько числом моделей, сколько тем, насколько модели **не похожи друг на друга**. Если базисы дают почти одинаковые ответы, усреднять нечего.

На практике это проявляется быстро, и кривая ошибки бэггинга падает с ростом K, а затем выходит на **плато**, после чего дальнейшее наращивание деревьев уже почти ничего не меняет.

Плато лежит ниже ошибки одиночного дерева, но его высота не зависит от того, сколько ещё деревьев готов обучить исследователь. Откуда берётся этот потолок качества и можно ли его поднять — главный вопрос части 1.

1.2. ДВА ИСТОЧНИКА СЛУЧАЙНОСТИ

Плато, на которое выходит кривая ошибки бэггинга, лежит тем выше, чем сильнее деревья похожи между собой. Чем ближе их ответы, тем меньше даёт усреднение. Бэггинг создаёт разнообразие единственным способом, выдавая каждому дереву собственную обучающую выборку, а случайный лес добавляет второй механизм, не зависящий от первого.

Первый источник — бутстрэп по объектам. Каждое дерево обучается на выборке размера N, набранной из исходной с возвращением (бутстрэп, bootstrap). Как считали в неделях 2 и 7, в такую выборку попадает примерно 63,2% уникальных объектов, а оставшиеся 36,8% в обучении этого дерева не участвуют и называются **out-of-bag** (OOB). Деревья видят разные данные и находят немного разные закономерности.

Второй источник — случайное подмножество признаков. Перед тем как разбить очередной узел, алгоритм отбирает случайные m признаков из p имеющихся и ищет лучший сплит **только среди них**, а остальные $p - m$ признаков в этом узле не рассматриваются.

Отбор проводится **заново перед каждым сплитом**, а не один раз на дерево, поэтому соседние узлы одного и того же дерева работают с разными наборами признаков.

Оба источника удобно посмотреть на сквозном примере лонгрида. Есть пять квартир, у каждой известны площадь, время до метро, этаж, год постройки и состояние ремонта, а предсказать нужно цену. На рисунке 2 показано, что достаётся каждому дереву и каждому узлу. Слева показано, какие объекты попали в бутстрэп-выборку каждого дерева, справа видно, какие признаки случайно выбраны в трёх узлах одного дерева.



Рисунок 2. Бутстрэп меняет состав объектов, а подмножество признаков выбирается случайно заново в каждом узле

ОПРЕДЕЛЕНИЕ

Случайный лес (random forest, RF) — ансамбль решающих деревьев, каждое из которых обучено на своей бутстрэп-выборке, причём в каждом узле оптимальный сплит ищется не по всем признакам, а только по случайно выбранному подмножеству из m признаков.

Если кратко: RF = бэггинг на деревьях + случайный выбор признаков на каждом сплите.

Метод предложил Лео Брейман, статью *Random Forests* он опубликовал в 2001 году; вместе с Адель Катлер он доводил метод и его программную реализацию.Брейман — автор бэггинга и случайного леса, а решающие деревья CART он разработал вместе с Джеромом Фридманом, Ричардом Олшеном и Чарльзом Стоуном.

Зачем нужна вторая случайность, лучше всего видно на ситуации, которую можно назвать монополией сильного признака. Пусть в данных есть один очень информативный признак и несколько умеренно информативных.

Тогда в бэггинге почти каждое дерево поставит сильный признак в корень, ведь, как бы ни различались бутстрэп-выборки, лучшим сплитом в корне почти всегда окажется одно и то же.

Верхушки деревьев получатся одинаковыми, а вслед за ними похожими станут и предсказания. Усреднение сильно скоррелированных величин снижает разброс куда слабее, чем усреднение независимых, так что бэггинг выигрывает у одиночного дерева совсем немного (см. рисунок 3).

Случайный выбор признаков устроен так, чтобы этого не происходило. Если в каждом узле доступны только m признаков из p, то в среднем доля $(p - m)/p$ сплитов сильный признак вообще не рассматривает, и решение приходится принимать по остальным. При $p = 100$ и $m = 10$ это девять сплитов из десяти.

У остальных признаков появляется возможность попасть в сплит, деревья перестают повторять друг друга, а среднее по ним становится устойчивее. Такой приём принято называть **декорреляцией** деревьев.



Рисунок 3. Если сильный признак доступен всегда, он оказывается в корне у всех деревьев сразу

ОПРЕДЕЛЕНИЕ

Декорреляция деревьев — сознательное снижение корреляции ρ между базовыми моделями за счёт дополнительной случайности при обучении, даже ценой ухудшения каждой модели по отдельности. В случайном лесу источником такой случайности служит выбор m признаков из p перед каждым сплитом.

Платить за это приходится качеством отдельного базиса, ведь дерево, которому в половине узлов не дали лучший признак, само по себе хуже дерева из бэггинга. Каждая модель ухудшается сознательно, ради того чтобы улучшить ансамбль целиком, и управляет этим обменом единственное число m:

- малое m — деревья почти независимы друг от друга, но каждое из них слабое;
- большое m — деревья сильные, но похожие друг на друга.

Удобную точку отсчёта задаёт крайний случай $m = p$, в котором никакого отбора не происходит, каждый сплит ищется по всем признакам и случайный лес превращается в точности в бэггинг на деревьях.

ВАЖНО

Случайный лес — это **не** метод случайных подпространств (random subspaces) из недели 7, хотя идея похожа. При методе случайных подпространств подмножество признаков фиксируется **один раз на всё дерево**, и, если сильный признак в него попал, он остаётся монополистом во всех узлах этого дерева, а если не попал, дерево не увидит его нигде.

В случайном лесу отбор идёт **в каждом узле заново**, поэтому один и тот же признак внутри одного дерева бывает и доступен, и недоступен в разных его ветвях.

1.3. КАК ОБУЧАЕТСЯ СЛУЧАЙНЫЙ ЛЕС

Обе идеи леса уже названы: бутстрэп по объектам и случайное подмножество признаков при выборе сплита. Ни та, ни другая не меняют алгоритм построения дерева, они лишь ограничивают то, что дереву показывают, поэтому внутри узла работает всё тот же перебор порогов из недели 6.

Осталось собрать эти идеи в процедуру по шагам и договориться, что делать с K готовыми ответами.

Алгоритм

Вход: обучающая выборка из N объектов с p признаками, число деревьев K, число признаков на сплите m, критерий останова.

Для $k = 1, \dots, K$:

1. Сформировать бутстрэп-выборку Z_k: N объектов, взятых из обучающей выборки с возвращением.
2. Построить дерево b_k по выборке Z_k. Пока не выполнен критерий останова, для каждого текущего узла повторять:
   - выбрать случайное подмножество из m признаков среди всех p;
   - перебрать только этих m кандидатов и найти среди них лучший признак и лучшую точку разбиения;
   - разделить узел на два дочерних.
3. Растить дерево глубоким и не обрезать его.

Выход: набор деревьев $b_1, \dots, b_K$ и правило агрегации.

Отдельно стоит оговорить требование растить деревья глубокими. Усреднение уменьшает разброс, но со смещением ничего не делает: среднее одинаково смещённых предсказаний остаётся смещённым. Поэтому от базовой модели выгоднее требовать низкого смещения, а её высокий разброс ансамбль возьмёт на себя. Именно поэтому неглубокие деревья здесь не подходят: они дают меньше разброса, но ценой большего смещения, которое усреднение неспособно устранить.

Ответ готового ансамбля в регрессии получается усреднением ответов деревьев:

$$a(x) = \frac{1}{K} \sum_{k=1}^{K} b_k(x).$$

В классификации простейший вариант — **голосование большинством** (majority vote), при котором каждое дерево называет класс, а ансамбль выбирает самый частый ответ:

$$a(x) = \text{mode}\{b_1(x), \dots, b_K(x)\}.$$

Когда нужны не метки, а вероятности, применяют второй вариант, в котором каждое дерево оценивает вероятность класса c долей объектов этого класса в листе, куда попал x, а такие оценки затем усредняются:

$$P(y = c \mid x) = \frac{1}{K} \sum_{k=1}^{K} P_k(y = c \mid x).$$

Эти два правила не эквивалентны, потому что голосование видит только итоговую метку дерева, а усреднение вероятностей учитывает ещё и уверенность: лист с перевесом девять к одному весит больше, чем лист с перевесом в один объект. В scikit-learn `RandomForestClassifier` по умолчанию усредняет вероятности и лишь затем берёт класс с наибольшей средней.

Весь алгоритм от исходных данных до ответа умещается в одну схему (см. рисунок 4).

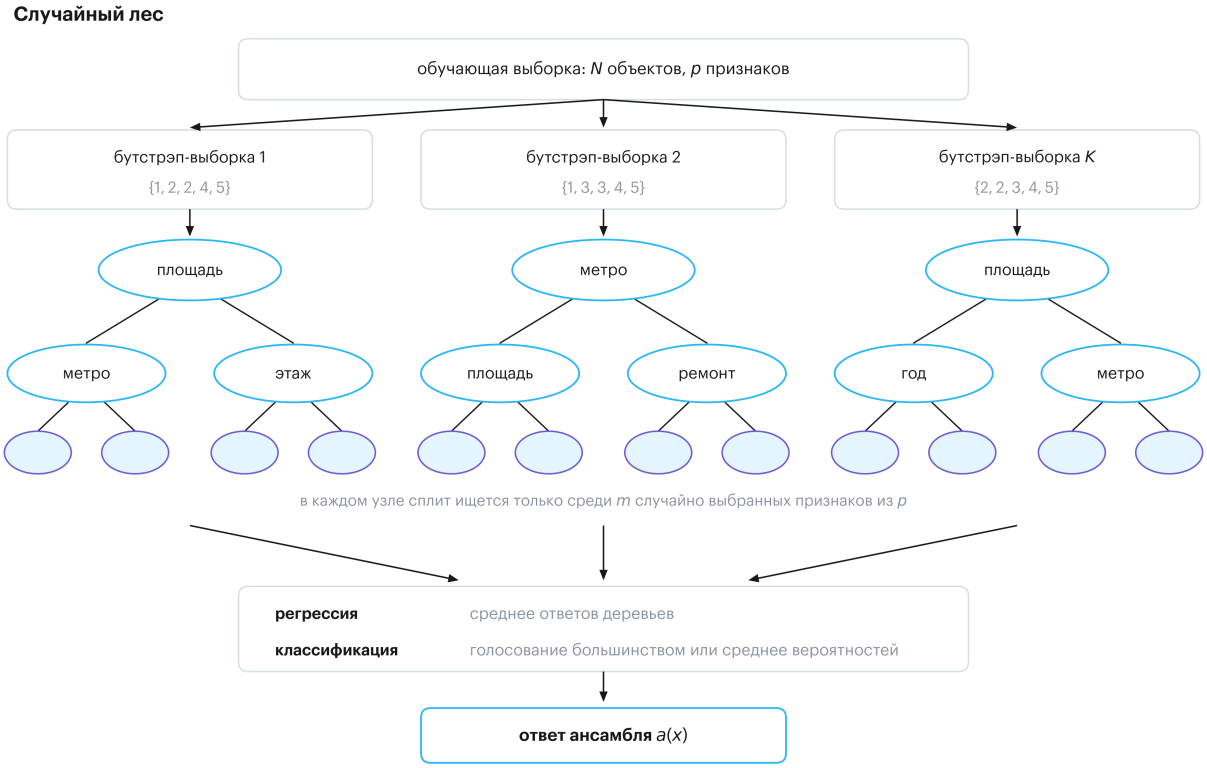


Рисунок 4. Обучающая выборка, K бутстрэп-выборок, K деревьев и одна агрегация на выходе

1.4. КАК СЛУЧАЙНОСТЬ СНИЖАЕТ КОРРЕЛЯЦИЮ

Рандомизация обходится дорого, потому что каждое отдельное дерево от неё становится слабее. Что получено взамен, показывает формула разброса параллельного ансамбля, выведенная в неделе 7:

$$\mathrm{Var}[a(x)] = \rho\sigma^2 + \frac{1-\rho}{K}\,\sigma^2.$$

- σ^2 — дисперсия ответа одного дерева в точке x; показывает, насколько сильно ответ этого дерева зависит от случайности обучения;
- ρ — корреляция ответов двух разных деревьев в той же точке; чем она выше, тем больше деревья повторяют друг друга;
- K — число деревьев в ансамбле.

Формула объясняет, почему добавление новых деревьев в какой-то момент почти перестаёт помогать. Она верна не всегда, а при трёх предположениях о базовых моделях:

- все b_k одинаково распределены (независимости при этом не требуется),
- у каждой из них одна и та же дисперсия σ^2,
- любая пара деревьев имеет одну и ту же корреляцию ρ.

Все три величины относятся к фиксированной точке x и в общем случае от неё зависят. Здесь и далее этот аргумент опускается для краткости.

В правой части формулы два слагаемых, которые по-разному зависят от числа деревьев:

слагаемое	зависит от K	что за ним стоит	что с ним делает усреднение
первое, $\rho\sigma^2$	нет, одинаково при десяти деревьях и при десяти тысячах	ковариационная часть дисперсии среднего: то, в чём ответы деревьев повторяют друг друга	воспроизводит, а не подавляет
второе, $\frac{1-\rho}{K}\sigma^2$	да, убывает к нулю с ростом K	доля разброса, которая у каждого дерева своя	гасит, и тем сильнее, чем больше деревьев

Различие между строками и задаёт всё поведение ансамбля, ведь наращивание K работает только со второй строкой, а первая от него не зависит.

Центральный факт про параллельные ансамбли состоит в том, что при этих предположениях разброс не опустится ниже $\rho\sigma^2$, сколько деревьев ни добавляй, то есть **нижнюю границу разброса задаёт не количество деревьев, а их похожесть**. Наращивание K эту границу не двигает, а двигают корреляция деревьев и разброс каждого из них. Полная ошибка вдобавок зависит от смещения, которое усреднение не уменьшает.

Насколько велика разница между этими двумя слагаемыми, видно на числах.

ПРИМЕР

Возьмём $\sigma^2 = 1$ и $K = 100$. Для бэггинга с $\rho = 0{,}7$ формула даёт $0{,}7 + 0{,}3/100 = 0{,}703$, а для леса с $\rho = 0{,}3$ она даёт $0{,}3 + 0{,}7/100 = 0{,}307$, то есть разброс упал более чем вдвое, хотя деревьев в обоих случаях поровну.

Для сравнения, увеличение числа деревьев в бэггинге со ста до десяти тысяч меняет $0{,}703$ на $0{,}70003$, то есть не даёт почти ничего.

Обе точки отмечены на рисунке 5 при $K = 100$: верхняя на кривой бэггинга, нижняя на кривой случайного леса.

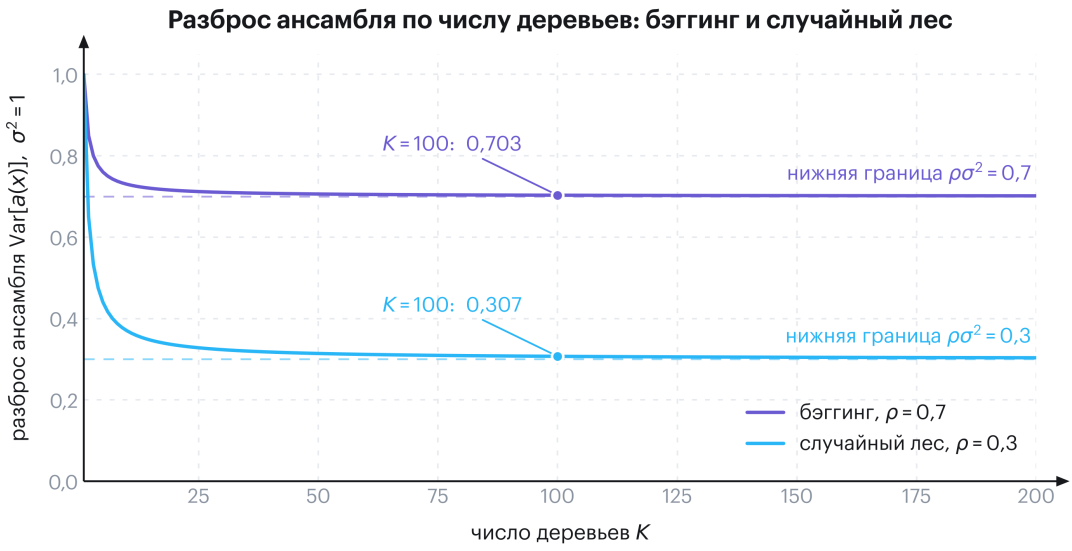


Рисунок 5. Второе слагаемое убывает с ростом K, первое от K не зависит вообще

1.4.1. Почему m не стоит доводить до единицы

Раз низкая корреляция настолько ценна, возникает вопрос, не взять ли в каждом узле один случайный признак, чтобы деревья разошлись максимально сильно.

Трудность в том, что m управляет сразу несколькими величинами, и меняются они в разные стороны. Уменьшение m действует сразу по трём направлениям:

- снижает ρ, и это желаемый эффект;
- увеличивает σ^2 отдельного дерева, ведь его структура всё сильнее зависит от случайного отбора;
- увеличивает смещение отдельного дерева, ведь информативный признак всё чаще оказывается недоступен в том узле, где он был бы лучшим.

Первые два эффекта входят в формулу как сомножители одного произведения и частично компенсируют друг друга, так что $\rho\sigma^2$ от уменьшения m может как упасть, так и вырасти. Третий эффект в формулу для разброса не входит, но вредит сильнее всего, поскольку смещение при усреднении сохраняется.

Проще всего это увидеть на разреженных данных. Дереву, которому в каждом узле доступен один случайный признак из двухсот, трудно найти закономерность, если информативных признаков всего два или три, а остальные шумовые.

Оба эффекта видны на прогоне (см. рисунок 6). Данные разреженные: пятьдесят признаков, из них информативных десять. Корреляция ответов деревьев растёт с m монотонно, от $0{,}013$ при $m = 1$ до $0{,}313$ при $m = 50$, а ошибка ведёт себя иначе: при $m = 1$ она равна $0{,}261$, минимума $0{,}155$ достигает при $m = 5$ и дальше снова растёт.

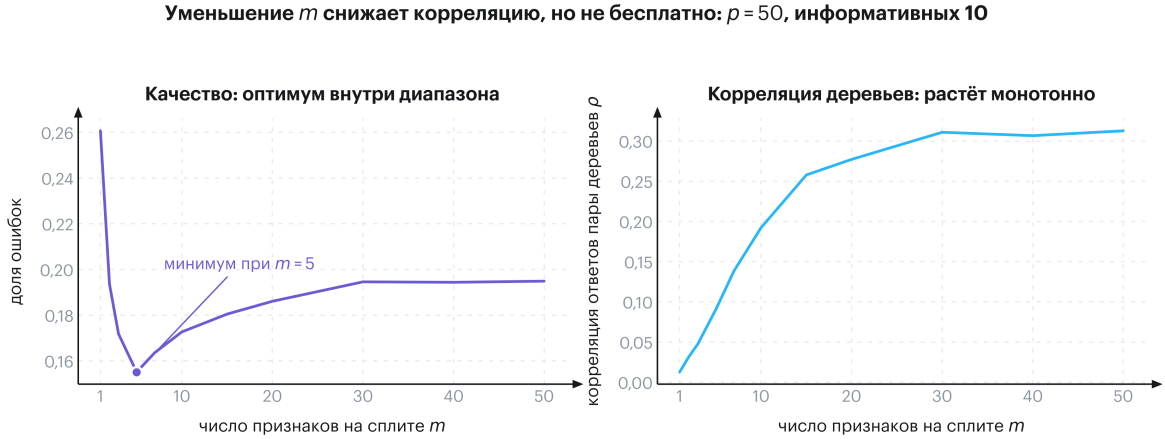


Рисунок 6. Корреляция деревьев падает с уменьшением m монотонно, а качество имеет оптимум внутри диапазона

Поэтому снижение ρ оказывается не самоцелью: параметр m влияет и на разнообразие деревьев, и на качество каждого из них по отдельности, а равновесие ищется между этими эффектами. Разумные значения по умолчанию разобраны в пункте 1.6.

INTERVIEW KIT

Разброс параллельного ансамбля не опускается ниже $\rho\sigma^2$. Число деревьев убирает только слагаемое $\frac{1-\rho}{K}\sigma^2$, поэтому после нескольких сотен деревьев рост K почти ничего не даёт. Нижнюю границу сдвигает лишь снижение корреляции, и главный рычаг здесь — m: чем меньше признаков доступно в узле, тем меньше деревья повторяют друг друга.

1.5. СЧИТАЕМ ЛЕС ВРУЧНУЮ

Разговор о корреляции базисов становится конкретным, если один раз построить лес вручную и довести все вычисления до конца.

Возьмём пять квартир, два признака (площадь x_1 и время до метро x_2) и будем предсказывать цену.

№	площадь x_1, м²	до метро x_2, мин	цена y, млн ₽
1	30	5	6
2	40	8	8
3	50	25	7
4	60	3	12
5	70	10	12

Лес будет состоять из $K = 3$ деревьев, и каждое из них будет пнём (stump), то есть сделает ровно одно разбиение. В этом единственном узле дереву доступен один случайный признак ($m = 1$ из $p = 2$). Пень выбран затем, чтобы весь счёт помещался в таблицу и делался руками. Настоящий лес растит деревья глубокими, и причина этого разобрана в разделе 1.3.

Качество сплита измеряется суммой квадратов отклонений от среднего внутри листа (SSE, sum of squared errors), сложенной по двум листьям. У пня сплит один, поэтому отбор признаков проводится один раз на дерево, тогда как в глубоком дереве он повторялся бы в каждом узле.

1.5.1. Как строятся три дерева леса

Бутстрэп дал набор **{1, 2, 2, 4, 5}**: объект 3 в выборку не попал, объект 2 попал дважды. Случайный отбор в корне выделил площадь x_1.

Кандидаты в пороги — середины между соседними значениями x_1, которые встретились в выборке (30, 40, 60, 70), то есть 35, 50 и 65.

порог	левый лист	правый лист	SSE
$x_1 \le 35$	$\{6\}$, среднее 6	$\{8, 8, 12, 12\}$, среднее 10	$0 + 16 = 16$
$x_1 \le 50$	$\{6, 8, 8\}$, среднее 7,33	$\{12, 12\}$, среднее 12	$2{,}67 + 0 =$ **2,67**
$x_1 \le 65$	$\{6, 8, 8, 12\}$, среднее 8,5	$\{12\}$, среднее 12	$19 + 0 = 19$

Побеждает порог 50 с большим отрывом:

$$b_1(x) = \begin{cases} 7{,}33, & x_1 \le 50 \\ 12, & x_1 > 50 \end{cases}$$

В листе стоит среднее цен тех объектов бутстрэп-выборки, которые в него попали, причём дубликат объекта 2 учитывается дважды.

Остальные два дерева строятся так же, поэтому для них приводится только итог перебора.

дерево	бутстрэп-выборка	признак	порог	левый лист	правый лист	SSE
2	{1, 3, 3, 4, 5}	x_2	$x_2 \le 4$	12	8	22
3	{2, 2, 3, 4, 5}	x_1	$x_1 \le 55$	7,67	12	0,67

Второе дерево получилось заметно хуже: его SSE равна 22 против 2,67 у первого. Отбор выдал ему время до метро, а этот признак на наших данных сам по себе объясняет цену слабо.

Такое дерево в лесу не исправляют и не отбрасывают, потому что разнообразие базисов как раз и означает, что некоторым из них достаётся неудачный набор признаков.

1.5.2. Что скажет лес про новую квартиру

На вход передаётся квартира площадью 55 м^2 в 12 минутах от метро, её истинная цена равна 10 миллионов рублей. Прогноз леса в регрессии — среднее прогнозов деревьев, поэтому ответы всех трёх нужно усреднить (см. рисунок 7).

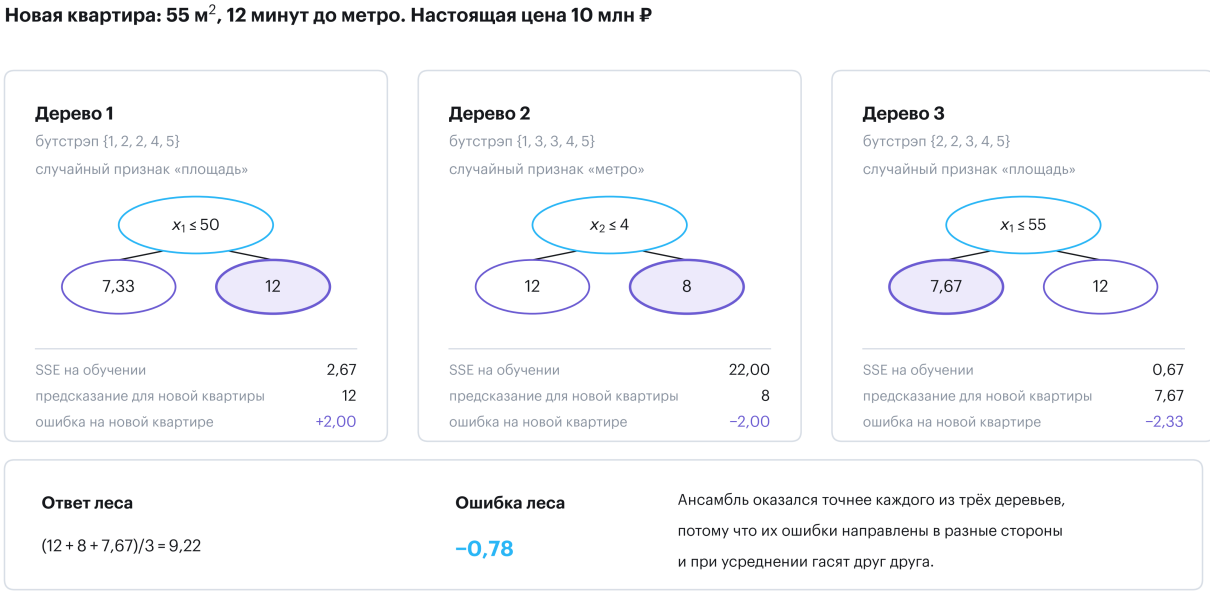


Рисунок 7. Каждый пень промахнулся примерно на два миллиона, а их среднее промахнулось на 0,78

Ошибка в таблице считается как предсказание минус истина, поэтому плюс означает переоценку, а минус — недооценку.

	правило	ответ	ошибка
дерево 1	$55 > 50$	12	$12 - 10 = +2{,}00$
дерево 2	$12 > 4$	8	$8 - 10 = -2{,}00$
дерево 3	$55 \le 55$	7,67	$7{,}67 - 10 = -2{,}33$
лес	среднее	$\frac{12 + 8 + 7{,}67}{3} = \mathbf{9{,}22}$	$9{,}22 - 10 = \mathbf{-0{,}78}$

В этих четырёх числах видна основная идея параллельного ансамбля. Каждое дерево промахнулось примерно на 2 миллиона, то есть по отдельности все три модели одинаково посредственны, но промахнулись они **в разные стороны**: первое переоценило квартиру, второе и третье недооценили.

Усреднение погасило разнонаправленные ошибки, и лес ошибся на 0,78, оказавшись точнее **каждого** своего дерева, а не просто точнее их среднего качества.

Обратный случай показывает, за счёт чего это произошло: если бы все три дерева промахнулись на $+2$, среднее промахнулось бы ровно на те же $+2$, и ансамбль не дал бы ничего. Выигрыш принесла разнонаправленность ошибок, а она и есть малое ρ из формулы для разброса.

INTERVIEW KIT

Ансамбль выигрывает потому, что модели ошибаются по-разному.

Разброс среднего складывается из двух частей: одна убывает с ростом числа моделей, вторая зависит только от того, насколько похожи ответы, и числом моделей не убирается.

В разобранном вручную лесу три пня ошиблись на $+2{,}00$, $-2{,}00$ и $-2{,}33$, а их среднее — на $-0{,}78$. Если бы все три промахнулись на $+2$, среднее промахнулось бы на столько же.

1.5.3. OOB-оценка: те же три дерева измеряют и качество

Объект обучающей выборки почти всегда удаётся предсказать теми деревьями, которые его не видели, и утечки при этом не возникает. Так получается OOB-оценка качества, для которой не нужны ни отложенная выборка, ни дополнительный проход по данным. Объекта 3 нет в первой выборке, объекта 2 во второй, объекта 1 в третьей.

объект	предсказывает	предсказание	истина	ошибка
1 (30 м^2, 5 мин)	дерево 3	7,67	6	$+1{,}67$
2 (40 м^2, 8 мин)	дерево 2	8	8	0
3 (50 м^2, 25 мин)	дерево 1	7,33	7	$+0{,}33$

$$\mathrm{MSE}_{\mathrm{OOB}} = \frac{1{,}67^2 + 0^2 + 0{,}33^2}{3} \approx 0{,}96.$$

Объекты 4 и 5 попали во все три выборки, поэтому OOB-предсказания для них не существует. При пяти объектах и трёх деревьях такое почти неизбежно, но, когда деревьев сотни, каждый объект оказывается out-of-bag примерно для трети из них и оценка становится достаточно устойчивой. Общая запись OOB-предсказания и ошибки по нему разобрана в неделе 7.

ОПРЕДЕЛЕНИЕ

OOB-оценка (out-of-bag estimate) — оценка качества леса по объектам, не попавшим в бутстрэп-выборку. Каждый объект предсказывают только те деревья, которые его не видели, поэтому OOB-оценку можно использовать вместо отдельной валидационной выборки. В sklearn при `oob_score=True` атрибут `oob_score_` содержит OOB-качество: по умолчанию долю верных ответов в классификации и R^2 в регрессии.

Как замена валидации при подборе гиперпараметров она работает хорошо, но итоговую цифру качества лучше получать на отложенной выборке, ведь все предсказания здесь сделаны на тех же данных, на которых лес обучался.

1.6. ГИПЕРПАРАМЕТРЫ

Осталось понять, какими параметрами лес настраивается и какие из них действительно что-то меняют. Настраивать приходится немного: большинство гиперпараметров лес унаследовал от одиночного дерева, а значения по умолчанию известны заранее и на большинстве задач работают.

гиперпараметр	за что отвечает	по умолчанию в sklearn	с чего начинать
`n_estimators`	число деревьев K	100	300 и больше, сколько позволяет бюджет
`criterion`	мера неоднородности, по которой выбирается сплит	`gini` в классификации, `squared_error` в регрессии	значение по умолчанию
`max_features`	сколько признаков m доступно на одном сплите	`'sqrt'` у классификатора, `1.0` (все признаки) у регрессора	$\sqrt{p}$ в классификации, $p/3$ в регрессии
`max_depth`	предел глубины дерева	`None`, то есть без ограничения	значение по умолчанию
`min_samples_split`	при каком размере узла его ещё дробят	2	значение по умолчанию
`min_samples_leaf`	минимальный размер листа	1	1 в классификации, около 5 в регрессии
`bootstrap`	обучать ли каждое дерево на бутстрэп-выборке из N объектов	`True`	значение по умолчанию
`oob_score`	считать ли out-of-bag-оценку	`False`	`True`, если отдельной валидации нет
`n_jobs`	сколько ядер занять под обучение	`None`, то есть одно ядро	`-1`, деревья независимы
`random_state`	фиксация бутстрэпа и случайного отбора признаков	`None`	любое фиксированное число

Вторая колонка — значения из сигнатуры `RandomForestClassifier` и `RandomForestRegressor`, третья — стартовые значения, с которых удобно начинать подбор. Значение `min_samples_leaf` около пяти в регрессии — эвристика, а не значение по умолчанию.

В коде это выглядит так — и для регрессии тоже, ведь в `RandomForestRegressor` набор параметров тот же.

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=300,
    criterion='gini',        # 'entropy', 'log_loss'
    max_features='sqrt',     # 'log2', число, доля или None
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    bootstrap=True,
    oob_score=True,          # требует bootstrap=True
    n_jobs=-1,
    random_state=0,
)
model.fit(X_train, y_train)

model.oob_score_             # OOB-оценка качества, посчитана без отдельной валидации
```

Дальше разобраны четыре из них, которые действительно приходится настраивать.

1.6.1. Количество деревьев, `n_estimators`

Деревья обучаются независимо и входят в среднее наравне друг с другом, поэтому рост K ансамбль не усложняет. **Рост `n_estimators` обычно не ухудшает качество систематически**: кривая ошибки выходит на плато и дальше почти не меняется. Строгой монотонности от конечной валидационной кривой при этом ждать не стоит, небольшие колебания на ней возможны.

Форма кривой на прогоне такая (см. рисунок 8): ошибка падает с $0{,}28$ при одном дереве до $0{,}15$ к пятидесяти, а дальше меняется в третьем знаке. OOB-оценка идёт рядом с ошибкой на отложенной выборке, $0{,}148$ против $0{,}151$ при ста деревьях, поэтому по ней и следят за выходом на плато. Начинается она с десяти деревьев, и причина здесь содержательная.

OOB-предсказание объекта складывается только из тех деревьев, в бутстрэп-выборку которых он не попал, а попадает он туда с вероятностью около $0{,}632$. Значит, при $K = 1$ объект остаётся без OOB-предсказания примерно в двух случаях из трёх.

С ростом числа деревьев доля таких объектов падает быстро: при $K = 3$ она около $0{,}25$, при $K = 10$ — около процента. Пока она велика, оценка считается по случайной части выборки, а не по всей, и от прогона к прогону скачет.

Тот же эффект виден на ручном примере с тремя деревьями: объекты 4 и 5 попали во все три бутстрэп-выборки и OOB-предсказания не получили. К этому добавляется поведение библиотеки: sklearn засчитывает непредсказанному объекту первый класс из `classes_` и сопровождает это предупреждением. Поэтому график и не показывает голубую кривую раньше десяти деревьев.

Цена у роста всё же есть. Обучение сотен глубоких деревьев стоит времени и памяти даже при `n_jobs=-1`.

Высоту плато задаёт разложение дисперсии из пункта 1.4: от числа деревьев зависит только второе слагаемое, а первое, $\rho\sigma^2$, остаётся на месте.

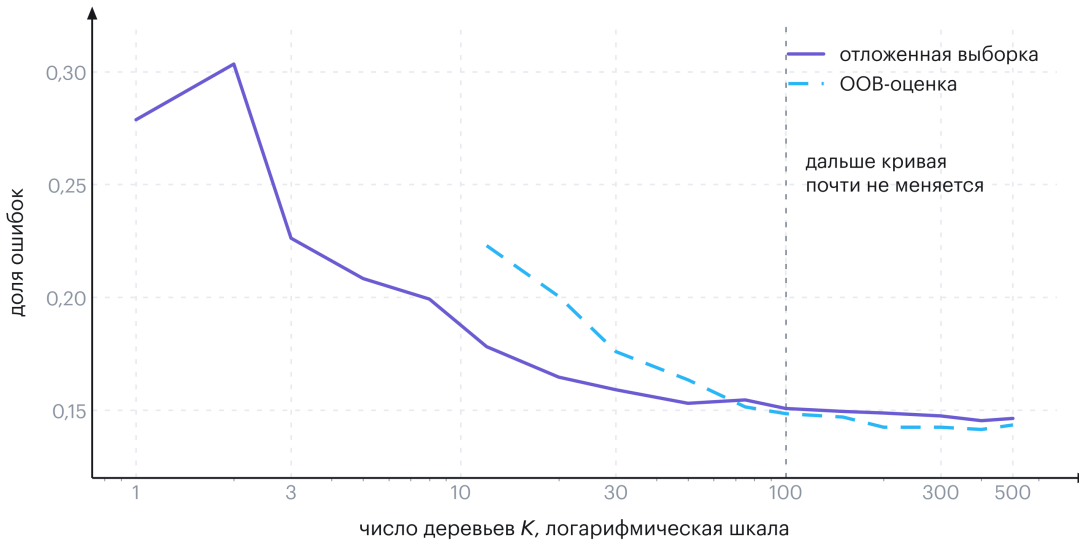


Рисунок 8. Ошибка выходит на плато, а OOB-оценка идёт рядом с отложенной выборкой. OOB-кривая начинается с $K = 10$: при меньшем числе деревьев оценка определена не для всех объектов

Практический вывод. Брать K с запасом и добавлять деревья, пока ошибка не вышла на плато. Следить за этим можно бесплатно, по OOB-оценке, без отдельной валидационной выборки. Для бустинга рассуждение уже не работает, потому что там число моделей становится одним из главных регуляризаторов (часть 2).

1.6.2. Мера неопределённости, `criterion`

Чем различаются критерий Джини и энтропия, подробно разобрано в неделе 6, а на качество леса выбор между ними влияет слабо, поскольку сплиты они поощряют почти одни и те же. В регрессии по умолчанию берут MSE, которая считается быстрее MAE, хотя и чувствительнее к выбросам. Разумно обучаться по той мере, по которой качество затем и оценивается.

1.6.3. Сложность деревьев, `max_depth`, `min_samples_split`, `min_samples_leaf`

Усреднение уменьшает разброс, а смещение одинаково распределённых деревьев переходит в ансамбль без изменений, поэтому от базовой модели выгоднее требовать низкого смещения.

Правилом без исключений это не становится: уменьшая `max_features`, смещение отдельного дерева повышают сознательно (пункт 1.4.1).

Низкое смещение у дерева означает подробное разбиение пространства признаков, то есть большую глубину. Отсюда и стандартная связка: `max_depth=None`, `min_samples_leaf=1` в классификации и `min_samples_leaf≈5` в регрессии.

Отдельное дерево при этом переобучается (overfitting), и это нормально, ведь переобучение проявляется как высокий разброс, а разброс усреднение уменьшает.

ВАЖНО

Утверждение «случайный лес не переобучается» слишком сильное. На зашумлённых данных, где велика доля ошибочных меток, очень глубокие деревья запоминают шум, и усреднение снимает лишь часть эффекта.

Насколько при этом помогают ограничители, видно на прогоне с искусственно испорченными метками (см. рисунок 9). До трёх десятых испорченных меток неограниченные деревья остаются лучшим вариантом: при чистых данных $0{,}147$ против $0{,}221$ у `max_depth=6`, при тридцати процентах шума $0{,}206$ против $0{,}215$. Выигрывают ограничители только при сорока процентах, и выигрыш небольшой: $0{,}279$ у ограниченной глубины против $0{,}302$ без ограничений.

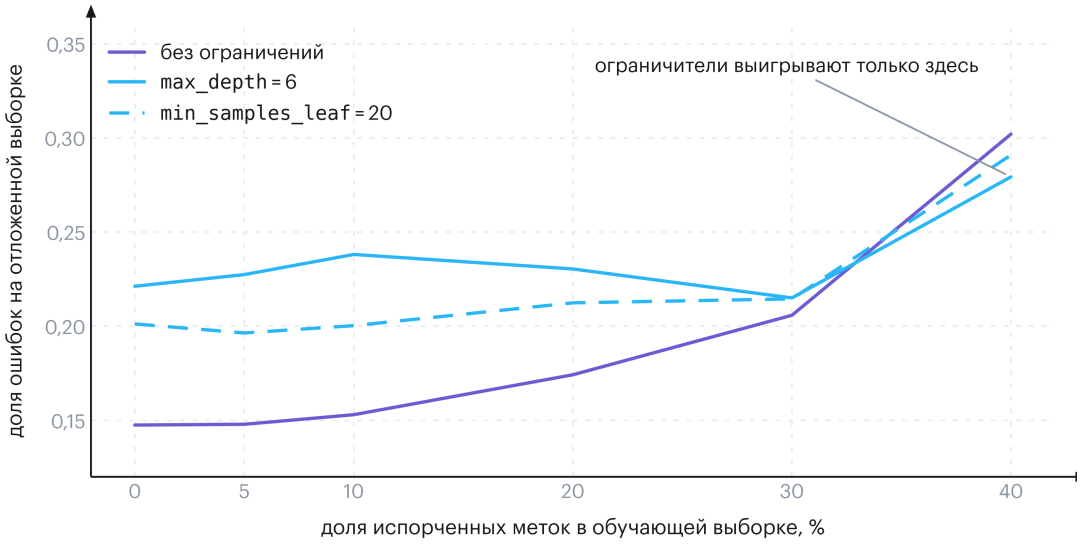


Рисунок 9. В этом эксперименте ограничители сложности помогли при сильном шуме в метках

Практический вывод. Растить деревья глубоко. Ограничители подключать при нехватке времени и памяти, а на шумных данных — проверять по валидации: выигрыш появляется только при большой доле ошибочных меток и обычно невелик.

1.6.4. Число признаков при сплите, `max_features`

На корреляцию деревьев этот параметр влияет сильнее прочих, а вместе с ней и на нижнюю границу, в которую упирается кривая ошибки.

Ловушка разобрана в пункте 1.4.1 и показана на рисунке 6: корреляция деревьев по m монотонна, а качество имеет оптимум внутри диапазона. Уменьшение m снижает корреляцию, но повышает и разброс отдельного дерева, и, что хуже, его смещение. Особенно заметно это в двух ситуациях:

- когда $m \ll p$ и дереву в каждом узле достаётся слишком мало признаков, чтобы выбрать осмысленный сплит;
- когда полезных признаков намного меньше, чем шумовых, и это тот самый случай с двумя-тремя информативными переменными из двухсот, где в случайный набор из m штук ни одна полезная чаще всего не попадает.

Отправные точки — $m = \sqrt{p}$ для классификации и $m = p/3$ для регрессии. Это старт, а не оптимум, и под конкретную задачу m подбирают отдельно, удобнее всего по OOB-оценке.

ВАЖНО

В sklearn `max_features='sqrt'` стоит по умолчанию только у `RandomForestClassifier`. У `RandomForestRegressor` по умолчанию берутся все p признаков, то есть без явного указания получается бэггинг, а не случайный лес.

1.6.5. Размер выборки на дерево, `max_samples` и `bootstrap`

Параметр `max_samples` задаёт долю обучающей выборки, которая идёт на одно дерево при `bootstrap=True`. По умолчанию он равен `None`, и тогда каждое дерево получает бутстрэп-выборку размера N, то есть столько же объектов, сколько в исходных данных.

Уменьшение доли даёт тот же обмен, что и уменьшение `max_features`, только через объекты, а не через признаки. Деревья видят меньше общих объектов, корреляция их ответов падает, обучение ускоряется, но каждое дерево при этом слабеет.

На прогоне минимум ошибки приходится не на полную выборку (см. рисунок 10): при доле $0{,}075$ ошибка составляет $0{,}107$ против $0{,}147$ на полной выборке, а OOB-оценка идёт рядом с отложенной по всему диапазону.

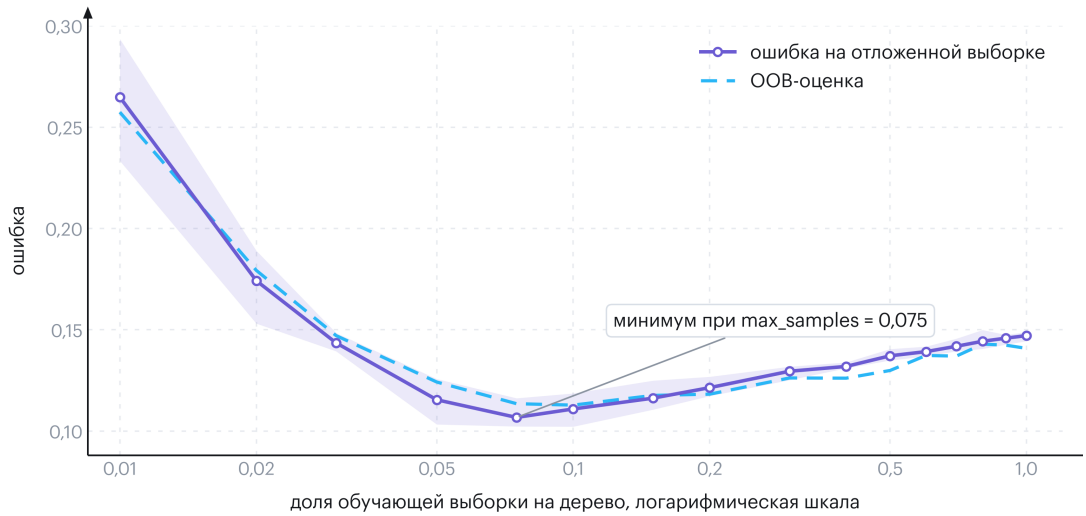


Рисунок 10. Уменьшение доли выборки на дерево снижает ошибку до некоторого предела, дальше дерево слабеет быстрее, чем падает корреляция

При `bootstrap=False` бутстрэпа нет вовсе: каждое дерево обучается на всей выборке, и различие между деревьями остаётся только в случайном отборе признаков. OOB-объектов при этом не возникает, поэтому OOB-оценка недоступна, и `oob_score=True` вместе с `bootstrap=False` вызывает ошибку.

Практический вывод. Параметр трогают прежде всего ради скорости на больших выборках. На качество он влияет слабее, чем `max_features`, но на зашумлённых данных проверить меньшую долю стоит: минимум ошибки может оказаться далеко от полной выборки.

1.7. ВАЖНОСТЬ ПРИЗНАКОВ

Оба способа измерить вклад признака (feature importance) разобраны для одиночного дерева в неделе 6, разделах 7.6 и 7.7. Краткое напоминание:

- **MDI** (mean decrease in impurity) складывает уменьшения неоднородности по всем узлам, где признак участвовал в сплите, так что чем сильнее упала суммарная неоднородность, тем важнее признак;
- **permutation importance** перемешивает значения признака между объектами и смотрит, насколько от этого вырастет ошибка уже обученной модели.

В ансамбле меняются не сами процедуры, а то, на чём они считаются, и обе от этого выигрывают.

MDI одного дерева — шумная величина, ведь стоит другому признаку выиграть сплит в корне, и вклады перераспределяются. В лесу MDI считается для каждого дерева отдельно и усредняется по всем K деревьям, включая те, где признак ни разу не использовался и его вклад равен нулю.

Деревья обучены на разных бутстрэп-выборках и видели разные наборы из m признаков, поэтому усреднение гасит случайность отдельных сплитов, и ранжирование получается заметно устойчивее.

Permutation importance выигрывает по другой причине. Посчитанная на обучающих данных, она отвечает на другой вопрос и даёт оптимистичную оценку, поэтому признак перемешивают на отложенной выборке. В лесу естественным источником такой выборки служат out-of-bag-объекты, но готовой функции для этого в sklearn нет: `permutation_importance` считает важность на той выборке, которую ей передали.

Обе оценки при этом не нейтральны. MDI смещён в сторону непрерывных и высококардинальных признаков, у которых больше возможных порогов, а permutation importance занижает важность взаимозаменяемых коррелирующих признаков, потому что при перемешивании одного модель опирается на его дублёра.

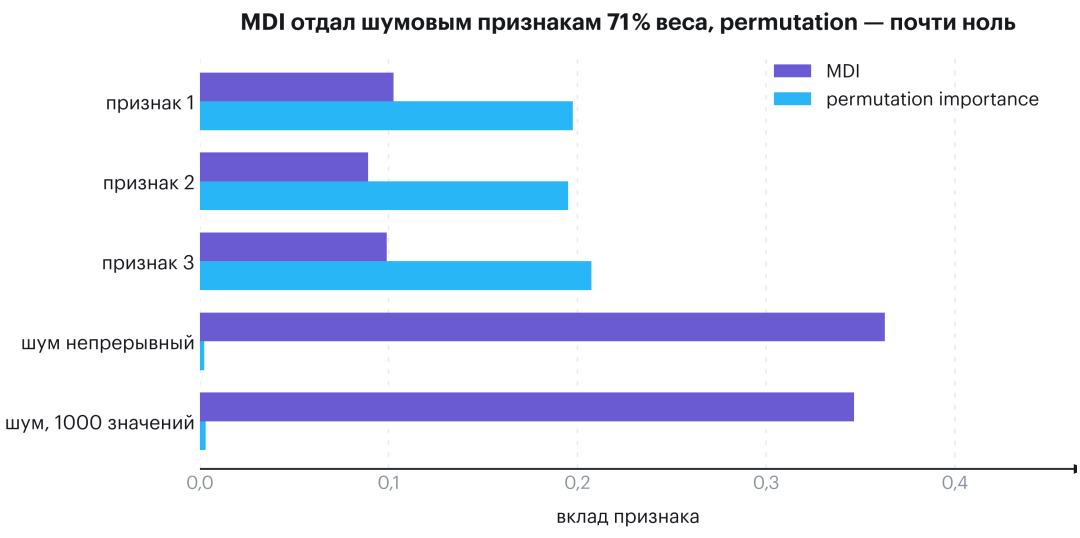


Рисунок 11. MDI отдал вес шумовым признакам с большим числом порогов, permutation importance — почти ноль

Смещение MDI хорошо видно на прогоне (см. рисунок 11). Метка зависит от трёх бинарных признаков, и на четверти объектов она испорчена, а к данным добавлены два заведомо шумовых признака: непре-

рывный и целочисленный с тысячей значений. MDI отдал этим двум 71% веса, тогда как permutation importance на отложенной выборке даёт им $0{,}002$ и $0{,}003$ против $0{,}20$ у настоящих признаков.

```python
from sklearn.inspection import permutation_importance

mdi = model.feature_importances_   # уже усреднён по всем деревьям леса
perm = permutation_importance(model, X_val, y_val,   # важность считается на отложенной
 ↪  выборке
                              n_repeats=10, random_state=0)
perm.importances_mean              # усреднение по n_repeats перестановкам
```

1.8. EXTRA TREES

Лес снижает корреляцию базисов двумя источниками случайности: бутстрэпом и подвыборкой признаков. **Extremely Randomized Trees (Extra Trees)** меняют набор источников случайности сразу в двух местах: добавляют случайный выбор порога и по умолчанию отказываются от бутстрэпа. Для каждого из m отобранных признаков алгоритм берёт **один случайный порог** из диапазона его значений в узле, и лучший сплит выбирается среди этих m кандидатов, а не среди всех возможных.

Extra Trees обучаются заметно быстрее, потому что перебор порогов — самая дорогая часть построения дерева, а деревья выходят ещё менее похожими друг на друга. Ценой становится рост смещения базовой модели, ведь удачный порог легко пропустить, но на многих задачах эта плата окупается более сильным падением разброса.

	случайный лес	Extra Trees
выборка для дерева	бутстрэп	вся выборка (`bootstrap=False` по умолчанию)
признаки на сплите	случайные m из p	так же
пороги-кандидаты	все пороги отобранных признаков	по одному случайному на признак
корреляция ρ	ниже, чем у бэггинга	ещё ниже
смещение базиса	как у полного дерева	обычно чуть выше
время обучения	точка отсчёта	в несколько раз меньше
OOB-оценка	доступна	только при `bootstrap=True`

Отказ от бутстрэпа — значение по умолчанию в sklearn, а не обязательное свойство метода. Параметр `bootstrap` можно включить, и тогда вернутся и бутстрэп-выборки, и OOB-оценка.

```python
from sklearn.ensemble import ExtraTreesClassifier

model = ExtraTreesClassifier(n_estimators=300, max_features='sqrt',
                             bootstrap=False, n_jobs=-1, random_state=0)
model.fit(X_train, y_train)
```

1.9. СИЛЬНЫЕ И СЛАБЫЕ СТОРОНЫ СЛУЧАЙНОГО ЛЕСА

Сильные стороны:

Свойство	Что это значит на практике
Сильный результат при небольшой настройке	На большинстве табличных задач параметры по умолчанию дают сильный baseline
Мало что нужно подбирать	Обычно настраивают `max_features` и глубину, число деревьев берут по бюджету
Бесплатная оценка ошибки	OOB-оценка получается попутно, отдельная кросс-валидация часто не нужна
Безразличие к шкалам	Нормализация не требуется, монотонные преобразования ничего не меняют
Устойчивые важности признаков	Считаются сразу после обучения и усреднены по сотням деревьев, но MDI смещён в сторону признаков с большим числом порогов, см. раздел 1.7
Параллелизация	Деревья независимы, обучение раскладывается по ядрам почти линейно (`n_jobs=-1`)

Ограничения:

Свойство	Что это значит на практике
Категориальные признаки	Реализации `RandomForest*` и `ExtraTrees*` в sklearn их не поддерживают, категории нужно закодировать заранее
Пропуски	Современные версии sklearn обрабатывают `NaN` прямо в деревьях, в более старых их приходится заполнять
Интерпретируемость прогноза	Важности признаков доступны, но путь от корня к конкретному ответу в ансамбле уже не проследить
Стоимость инференса	Память и время предсказания растут пропорционально K, на высокой нагрузке это может стать узким местом
Экстраполяция	Прогноз кусочно-постоянный, за пределы диапазона обучающих ответов лес не выходит
Разреженные высокоразмерные данные	Чаще проигрывает линейным моделям: среди случайных m признаков почти все нулевые
Смещение	При одинаково распределённых деревьях усреднение его не уменьшает, каким бы большим ни было K

Именно последнее ограничение и оказывается принципиальным. **Пока базовое дерево обладает низким смещением, лес даёт сильный результат, но там, где такую модель построить не получается, добавление деревьев смещения ансамбля не уменьшит.**

INTERVIEW KIT

Случайный лес — сильный вариант по умолчанию: часто даёт хороший результат при небольшой настройке, оценка обобщающей ошибки получается попутно по OOB, шкалы признаков не важны, обучение раскладывается по ядрам. Отказываются от него ради интерпретируемости отдельного прогноза, ради скорости инференса или потому, что базовые деревья систематически смещены. Первые две причины упираются в инженерные ограничения, третья в устройство самого ансамбля.

Из этого ограничения вырастает вопрос: что, если очередная модель будет не ещё одной случайной копией предыдущих, а будет строиться специально под то, что предыдущие объяснить не смогли? Тогда модели перестают быть взаимозаменяемыми, порядок их построения начинает иметь значение, и добавление новой уменьшает систематическую ошибку уже накопленного ансамбля. Этим и займёмся в части 2.

2. Градиентный бустинг

2.1. ИДЕЯ БУСТИНГА

Часть 1 закончилась на ограничении, которое усреднением снять нельзя. Часть 2 строит ансамбль иначе, и понять, в чём разница, проще всего не по формулам, а по простой аналогии.

Рассмотрим игру в гольф: обычно игрок не пытается попасть в лунку одним ударом. Первый удар делается с размаха, покрывает основное расстояние и почти наверняка проходит мимо. Второй короче, потому что до цели осталось меньше и бить надо аккуратнее, третий ещё короче. Последние удары уже подправляют промахи предыдущих, то перелетая лунку, то не докатываясь до неё.

Ни один удар в отдельности задачу не решает, а итоговое положение мяча определяется не лучшим ударом и не средним из ударов, а их суммой (см. рисунок 12).

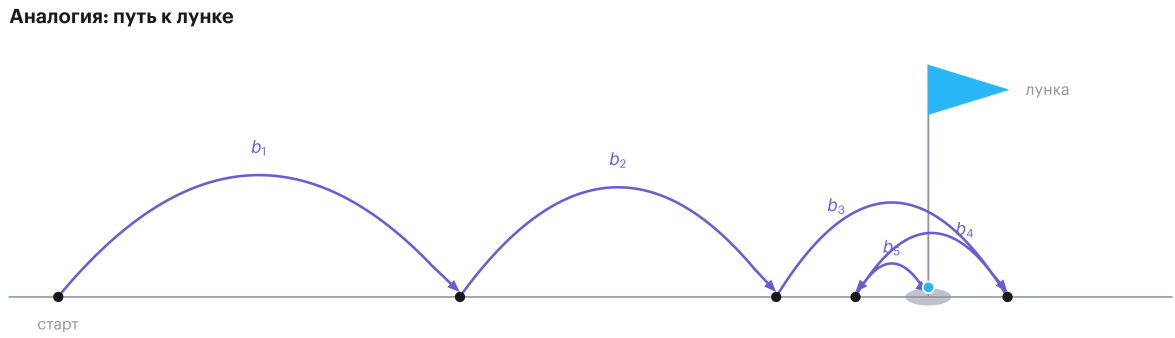


Рисунок 12. Итоговая позиция — сумма всех ударов, а не результат одного лучшего

Ограничение, которое мешает лесу двигаться в эту сторону, записывается одной строкой: смещение базовых моделей переходит в ансамбль без изменений.

$$\text{Bias}\big[a(x)\big] = \text{Bias}\big[b_k(x)\big].$$

Математическое ожидание среднего равно среднему математических ожиданий, а базовые модели в лесу обучаются по одному рецепту на данных из одного распределения и потому смещены одинаково. Тысяча таких деревьев даёт ровно то же смещение, что и одно.

Ломается схема там, где глубокое дерево не столько объясняет сигнал, сколько запоминает шум. Сильную базовую модель с низким смещением на таких данных не построить, а наращивание таких деревьев смещение ансамбля не уменьшит: у одинаково распределённых базисов оно переходит в среднее без изменений.

Значит, чтобы взяться за смещение, нужна конструкция с противоположным принципом, и аналогия из начала раздела подсказывает какая.

ОПРЕДЕЛЕНИЕ

Бустинг (boosting) — метод построения ансамбля, в котором несколько базовых моделей последовательно объединяются в одну более сильную. Каждая следующая модель строится с учётом ошибок предыдущих, благодаря чему ансамбль постепенно улучшает качество своих предсказаний.

Наиболее распространённый вариант бустинга сегодня — **градиентный бустинг (gradient boosting, GB)**. Ему и посвящена часть 2.

Отсюда следствие, ради которого всё и затевалось. Базовой модели не обязательно сразу хорошо решать всю задачу: достаточно, чтобы она была немного лучше константного предсказания, а следующая модель сосредоточится на ошибках, которые она оставила. Ограничение части 1, где качество ансамбля упиралось в смещение базовой модели, здесь снимается.

Проще всего сопоставить обе конструкции на одной картинке (см. рисунок 13).

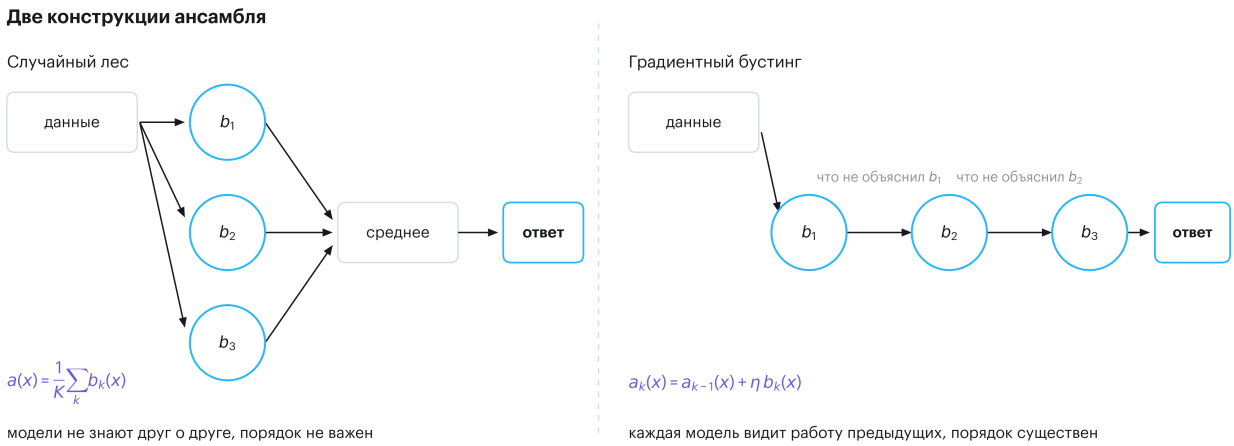


Рисунок 13. Слева модели обучаются независимо и усредняются, справа каждая следующая видит работу предыдущих

Ту же разницу видно и прямо в формулах. В неделе 7 ансамбль собирался усреднением, где базисы обучены независимо, входят с одинаковым весом и порядок их построения не важен:

$$a(x) = \frac{1}{K}\sum_{k=1}^{K} b_k(x).$$

Последовательный ансамбль разом не собирается, а накапливается по одному слагаемому. Есть грубое начальное приближение $a_0(x)$, играющее роль позиции, с которой начинал удар гольфист в нашем примере, а дальше на каждом шаге к уже накопленному прибавляется очередной базис:

$$a_k(x) = a_{k-1}(x) + b_k(x).$$

- $a_{k-1}(x)$ — ансамбль, накопленный за первые $k-1$ шагов;
- $b_k(x)$ — очередной базис, предсказывающий поправку к нему.

Записанная так формула содержит одну неприятность. Базис b_k подобран под конечную обучающую выборку, и вместе с полезным сигналом он неизбежно поймал какую-то долю шума в ней. Прибавляя базис целиком, ансамбль делает максимально жадный шаг по одной обучающей выборке. Следующие деревья, вообще говоря, могут скомпенсировать перебор, но траектория обучения от этого становится неустойчивой.

Приём против этого простой и состоит в том, чтобы не доверять одному базису всю оставшуюся ошибку. Поправка домножается на небольшое число η и входит в ансамбль укороченным шагом,

$$a_k(x) = a_{k-1}(x) + \eta\, b_k(x).$$

Множитель $\eta = 0{,}1$ даёт шаг в ту же сторону, но короткий, а остальное достаётся следующим базисам, которые пересчитают ошибку заново, уже видя результат сделанного шага. Темп обучения укорачивает шаг и тем самым сглаживает траекторию обучения, распределяя поправку по многим итерациям.

ОПРЕДЕЛЕНИЕ

Темп обучения η (learning rate, shrinkage) — множитель, с которым очередная базовая модель входит в ансамбль: $a_k = a_{k-1} + \eta\, b_k$. На практике берут небольшие положительные значения, чаще всего около $0{,}1$.

Платой становится число шагов, и чем меньше η, тем больше базисов нужно, чтобы пройти то же расстояние. Темп обучения и число базисов настраивают вместе.

Осталось понять, что именно должно стоять на месте b_k. Чтобы задачу вообще можно было записать, нужна мера расхождения — **функция потерь** $L(y, z)$, которая штрафует отклонение предсказания z от истинного ответа y на одном объекте.

Нужен базис, который сильнее всего уменьшит суммарные потери ансамбля после прибавления,

$$b_k = \arg\min_b \sum_{i=1}^{N} L\big(y_i,\ a_{k-1}(x_i) + b(x_i)\big).$$

- y_i — истинный ответ на i-м объекте;
- $a_{k-1}(x_i)$ — то, что ансамбль предсказывает на этом объекте сейчас;
- $b(x_i)$ — поправка, которую даст на нём новый базис.

Записать задачу оказалось легко, а решить её трудно. Во-первых, пространство поиска — множество всех функций, и перебрать его напрямую нельзя: точной формулы для минимума такая задача не имеет. Во-вторых, найденный базис должен предсказывать и за пределами обучающей выборки. Подобрать по одному числу на каждый из N объектов несложно, но такой набор чисел не скажет ничего о новой квартире, которой в обучающей выборке не было.

Оба затруднения снимает выбор базиса, и дерево удобно здесь сразу по трём причинам.

1. Оно кусочно-постоянно и потому задаёт поправку на всём пространстве признаков, а не только в обучающих точках.
2. Ему безразличны шкалы признаков, а значит, поправки не нужно приводить к общему масштабу.
3. И главное, оно умеет подстроиться почти под любой набор нужных поправок, если сгруппировать объекты с похожими поправками в один лист.

Градиентный бустинг над решающими деревьями обозначают **GBDT** (gradient boosting over decision trees). Именно этот вариант стоит за XGBoost, LightGBM и CatBoost, и дальше речь идёт только о нём.

У замены усреднения на сложение есть цена, и связана она с границами применимости разложения ошибки. Само разложение на смещение и разброс верно для любой модели, но формула разброса параллельного ансамбля выведена в предположении, что базисы одинаково распределены и что любая их пара имеет одну и ту же корреляцию ρ.

В бустинге не выполняется ни то ни другое, ведь модель b_k строится по ответам $b_1, \dots, b_{k-1}$ и вне их контекста смысла не имеет, а первое дерево и десятое делают совсем разную работу. Говорить о попарной корреляции базисов как о величине, которую понижают случайностью, здесь уже не приходится, и рычага ρ из части 1 в бустинге не будет.

2.2. СЧИТАЕМ БУСТИНГ ВРУЧНУЮ НА ЗАДАЧЕ РЕГРЕССИИ

Начинать удобнее с **частного случая, где задача регрессионная, а мера качества квадратичная**. Такой выбор не означает, что бустинг устроен именно так. Причина в другом. В этом случае каждая величина считается в одну строку, и виден каждый шаг.

То, что здесь выглядит как определение бустинга, окажется частным случаем: дальше схема обобщается на произвольную функцию потерь.

Данные — тех же пяти квартир, что и в примере с лесом, но признак остался один — площадь.

№	площадь x, м^2	цена y, млн ₽
1	30	6
2	40	8
3	50	7
4	60	12
5	70	12

Базовой моделью, как и в примере с лесом, будет пень, а темп обучения возьмём $\eta = 0{,}5$.

2.2.1. Шаг 0: одна константа на всех

Ансамбль должен с чего-то начинаться, и самое дешёвое приближение — константа, а для квадратичной ошибки лучшая константа равна среднему значению отклика:

$$a_0 = \bar{y} = \frac{6 + 8 + 7 + 12 + 12}{5} = 9.$$

Средний квадрат ошибки такой модели равен $6{,}4$, и это отправная точка для сравнения всего дальнейшего. Дальше понадобится величина, показывающая на каждом объекте по отдельности, насколько предсказание расходится с истинным значением. Это **остаток** (residual), знакомый по неделям 3–4, только вместо предсказания линейной регрессии здесь стоит текущий ансамбль:

$$r_i = y_i - a_{k-1}(x_i).$$

Остаток показывает, какой поправки не хватает предсказанию на этом объекте, и пересчитывается заново перед каждой итерацией. Для начального приближения $a_0 = 9$ остатки получаются такими:

№	x	y	a_0	расчёт	остаток r_i
1	30	6	9	$6 - 9$	-3
2	40	8	9	$8 - 9$	-1
3	50	7	9	$7 - 9$	-2
4	60	12	9	$12 - 9$	$+3$
5	70	12	9	$12 - 9$	$+3$

Три маленькие квартиры константа переоценивает, две большие недооценивает. В остатках осталась структура, и извлечь её должна следующая модель.

2.2.2. Итерация 1: первое дерево обучается на остатках от константы

Ключевой момент всего алгоритма — **дерево обучается не на цены, а на остатки**. Целевой переменной для первого дерева служит столбец r_i, а столбца y_i дерево не видит вообще.

Во всём остальном это обычное обучение пня: перебираются все пороги, и для каждого считается сумма квадратов отклонений внутри листьев (SSE), где предсказанием листа служит среднее попавших в него остатков.

порог	левый лист	правый лист	SSE
$x \le 35$	$\{-3\}$, среднее -3	$\{-1, -2, 3, 3\}$, среднее $0{,}75$	$0 + 20{,}75 = 20{,}75$
$x \le 45$	$\{-3, -1\}$, среднее -2	$\{-2, 3, 3\}$, среднее $1{,}33$	$2 + 16{,}67 = 18{,}67$
$x \le 55$	$\{-3, -1, -2\}$, среднее -2	$\{3, 3\}$, среднее 3	$2 + 0 = \mathbf{2}$
$x \le 65$	$\{-3, -1, -2, 3\}$, среднее $-0{,}75$	$\{3\}$, среднее 3	$20{,}75 + 0 = 20{,}75$

Порог $x \le 55$ выигрывает с большим отрывом, потому что он единственный отделяет отрицательные остатки от положительных, тогда как остальные три перемешивают их в одном листе. Первое дерево получается таким:

$$b_1(x) = \begin{cases} -2, & x \le 55 \\ +3, & x > 55 \end{cases}$$

К ансамблю прибавляется не всё предсказание дерева, а только его половина, потому что $\eta = 0{,}5$:

$$a_1(x) = a_0(x) + \eta\, b_1(x).$$

№	x	y	a_0	b_1	a_1	новый остаток
1	30	6	9	-2	8	-2
2	40	8	9	-2	8	0
3	50	7	9	-2	8	-1
4	60	12	9	$+3$	10,5	$+1{,}5$
5	70	12	9	$+3$	10,5	$+1{,}5$

Ошибка упала с $6{,}4$ до $1{,}9$, но в ноль остатки не обратились даже там, где дерево предсказало их точно: на объектах 4 и 5 дерево выдало $+3$, а в ансамбль вошла лишь половина, $+1{,}5$. Шаг укорочен намеренно: так отдельная неудачная поправка не фиксируется в ансамбле целиком, а остаток достаётся следующему дереву.

2.2.3. Итерации 2 и 3: каждое дерево обучается на остатках предыдущего

Процедура дальше повторяется без изменений, и каждое следующее дерево обучается на остатках, которые оставил предыдущий шаг. Поэтому полные таблицы перебора порогов для второй и третьей итераций опущены, а результат каждого шага зафиксирован в том же виде, что и на первой итерации.

Итерация 2. Второе дерево учится на столбце остатков, оставшемся после первого шага. Лучшим оказывается **тот же порог** $x \le 55$ с SSE 2, а значения в листьях равны -1 и $+1{,}5$, то есть вдвое меньше, чем у первого дерева.

№	x	y	a_1	b_2	a_2	новый остаток
1	30	6	8	-1	7,5	$-1{,}5$
2	40	8	8	-1	7,5	$+0{,}5$
3	50	7	8	-1	7,5	$-0{,}5$
4	60	12	10,5	$+1{,}5$	11,25	$+0{,}75$
5	70	12	10,5	$+1{,}5$	11,25	$+0{,}75$

Совпадение порогов здесь неслучайное, ведь половина первоначальной ошибки была оставлена неисправленной специально, и второму дереву досталась именно она, в том же виде и на том же месте. Ошибка ансамбля падает с $1{,}9$ до $0{,}775$.

Итерация 3. Третье дерево учится уже на новых остатках, и здесь разбиение **меняется**: побеждает порог $x \le 35$ с SSE $1{,}0625$, а листья получают значения $-1{,}5$ и $+0{,}375$.

№	x	y	a_2	b_3	a_3	новый остаток
1	30	6	7,5	$-1{,}5$	6,75	$-0{,}75$
2	40	8	7,5	$+0{,}375$	7,6875	$+0{,}3125$
3	50	7	7,5	$+0{,}375$	7,6875	$-0{,}6875$
4	60	12	11,25	$+0{,}375$	11,4375	$+0{,}5625$
5	70	12	11,25	$+0{,}375$	11,4375	$+0{,}5625$

Причина смены порога видна в столбце остатков. После первых двух деревьев основная зависимость цены от площади уже скорректирована, поэтому прежнее разбиение больше не даёт наилучшего разделения остатков. На объекте 1 накопилась невязка вдвое крупнее любой другой, и дерево отделяет именно его. Ошибка падает до $0{,}3531$.

Три дерева, каждое со своим порогом и своими значениями в листьях, показаны на рисунке 14 вместе с расчётом ответа ансамбля.



Рисунок 14. Ответ ансамбля — начальное приближение плюс поправки всех деревьев, взятые с коэффициентом η

2.2.4. Результат и выводы после ручного расчёта

объект	a_0	a_1	a_2	a_3	истина y
30 м^2	9	8	7,5	6,75	6
40 м^2	9	8	7,5	7,6875	8
50 м^2	9	8	7,5	7,6875	7
60 м^2	9	10,5	11,25	11,4375	12
70 м^2	9	10,5	11,25	11,4375	12
MSE	**6,4**	**1,9**	**0,775**	**0,3531**	

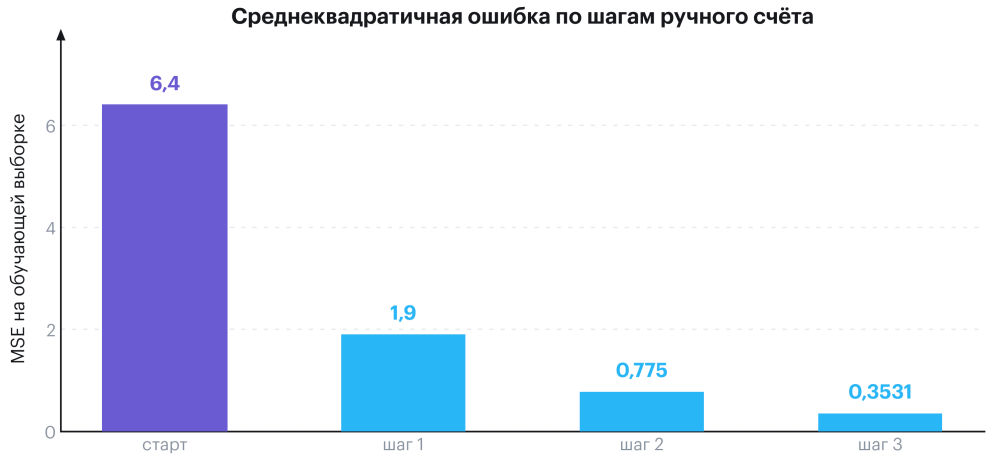


Рисунок 15. Ошибка падает в восемнадцать раз за три шага, хотя ни одно дерево по отдельности почти ничего не умеет

Ошибка упала примерно в восемнадцать раз за три шага, притом что каждое отдельное дерево здесь предельно примитивно и само по себе почти бесполезно. Из этого расчёта складываются четыре наблюдения об устройстве бустинга.

1. **При квадратичной ошибке дерево учится на остатках, а не на целевой переменной.** Признаки x_i остаются прежними, а целевой столбец пересчитывается заново перед каждой итерацией. Все три дерева решали формально одну и ту же задачу регрессии, просто с разными ответами.
2. **Темп обучения не даёт дереву закрыть ошибку целиком за один шаг.** Именно поэтому второе дерево повторило разбиение первого, а значения в его листьях оказались вдвое меньше. Ансамбль приближается к данным мелкими шагами, а не одним прыжком.
3. **Характер работы деревьев обычно меняется от шага к шагу.** Первые описывают крупную структуру, поздние уточняют детали. Третье дерево уже выделило объект 1 в собственный лист, и это фактор риска: при дальнейшем росте числа деревьев ансамбль начинает запоминать отдельные наблюдения.
4. **Систематическая ошибка уменьшается последовательно, а не усреднением.** Пень с двумя листьями по отдельности грубо смещён, но каждый следующий видит ошибку предыдущего, и сумма пней приближает зависимость точнее любого слагаемого. В случайном лесу из таких пней возможности ансамбля были бы сильно ограничены.

INTERVIEW KIT

В градиентном бустинге для квадратичной ошибки каждое следующее дерево обучается на остатках $r_i = y_i - a_{k-1}(x_i)$ и прибавляется к ансамблю с коэффициентом η. Ранние деревья описывают крупную структуру, поздние уточняют детали.

2.2.5. Алгоритм градиентного бустинга над деревьями для квадратичной ошибки

От шага к шагу в расчёте ничего не менялось, поэтому его можно записать сразу для произвольного числа итераций. Отдельной аккуратности требует только шаг 0, где среднее было взято не потому, что оно чем-то особенно хорошо, а потому, что среди всех констант именно оно доставляет минимум суммарной квадратичной ошибки. В общем виде начальное приближение — лучшая константа для выбранной функции потерь, и записывается оно соответственно.

Алгоритм

Градиентный бустинг над деревьями для квадратичной ошибки

Шаг 0. Взять лучшую константу: $a_0 = \arg\min_c \sum_{i=1}^{N} L(y_i, c)$. При квадратичной ошибке это $\bar{y}$.

Шаги $k = 1, \dots, K$.

1. Посчитать остатки $r_i^{(k)} = y_i - a_{k-1}(x_i)$ для всех $i = 1, \dots, N$.
2. Обучить дерево b_k на выборке $\{(x_i,\ r_i^{(k)})\}_{i=1}^{N}$, то есть на остатках вместо целевой переменной.
3. Обновить ансамбль: $a_k(x) = a_{k-1}(x) + \eta\, b_k(x)$.

Ответ: $a_K(x) = a_0 + \eta \sum_{k=1}^{K} b_k(x)$.

Те же шаги, развёрнутые на числах сквозного примера, собраны на рисунке 16. Номера шагов на схеме совпадают с номерами в блоке с алгоритмом, поэтому каждую строку алгоритма видно на конкретных числах: столбик остатков, дерево, которое на них обучается, и укороченная поправка, уходящая в ансамбль.

Такое пошаговое наращивание, когда слагаемые добавляются по одному, а уже добавленные больше не пересматриваются, называют **аддитивной моделью, построенной пошагово** (forward stagewise additive modeling). Термин подчёркивает главное свойство конструкции, ведь на шаге k первые $k-1$ слагаемых считаются окончательными и подбирается только очередное. Этим бустинг отличается от подгонки всех K деревьев сразу, которая была бы вычислительно неподъёмной, и отсюда же берётся его зависимость от порядка построения, которой в лесу нет.

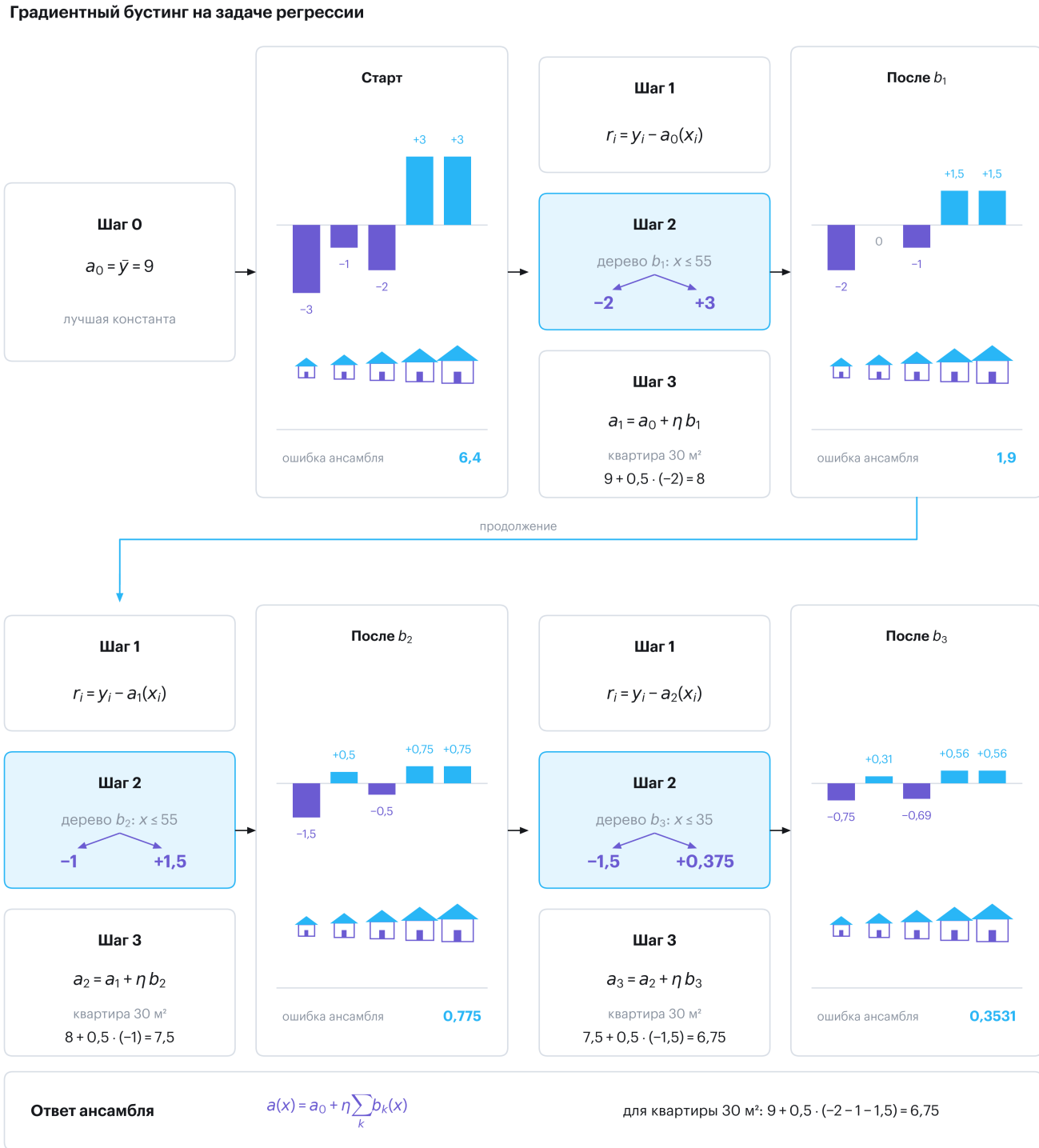


Рисунок 16. Алгоритм на числах сквозного примера. Голубые столбики — занижение ансамбля, фиолетовые — завышение

ВАЖНО

Алгоритм вышел проще, чем можно было ожидать, и у этого есть причина. До сих пор в расчётах стояла квадратичная ошибка, а у неё редкое свойство: нужная поправка выражается обычной разностью $y_i - a_{k-1}(x_i)$. У других функций потерь либо нет такой разности, либо она лишена содержательного смысла, поэтому принимать «бустинг обучается на остатках» за определение бустинга не стоит.

Классификация — как раз тот случай, где такая разность теряет смысл. Вычесть $y_i - a_{k-1}(x_i)$ формально можно, но метка и логит живут на разных шкалах, и содержательной поправкой эта величина не является. Осмысленная разность появляется только после дифференцирования, и она равна $y_i - p_i$. Поэтому следующий раздел обобщает алгоритм на произвольную функцию потерь.

2.3. ПРОИЗВОЛЬНАЯ ФУНКЦИЯ ПОТЕРЬ: ОТ ОСТАТКОВ К АНТИГРАДИЕНТУ

Вопрос этого раздела прямой. Что остаётся от схемы бустинга, когда квадратичной ошибки нет и разность $y_i - a_{k-1}(x_i)$ считать не из чего?

Пусть функция потерь $L(y, z)$ произвольная, и требование к ней одно: по второму аргументу, то есть по предсказанию, у неё должна существовать производная. Строгой дифференцируемости всюду при этом не требуется, достаточно, чтобы производная существовала почти всюду, а в отдельных точках её заменял субградиент. Сама задача при этом остаётся той, что поставлена в разделе 2.1. Нужен базис, после прибавления которого суммарные потери ансамбля упадут сильнее всего,

$$b_k \;=\; \arg\min_{b} \sum_{i=1}^{N} L\big(y_i,\; a_{k-1}(x_i) + b(x_i)\big).$$

Решать её в лоб не выходит. Минимум ищется по функции b, перебрать все функции нельзя, и входит b в сумму слишком сложно, чтобы взять производную и приравнять её к нулю.

Помогает **временное упрощение**: посмотреть на ансамбль не как на сумму, набираемую по шагам, а как на одну функцию a, которую надо подобрать целиком,

$$\sum_{i=1}^{N} L\big(y_i,\; a(x_i)\big) \;\longrightarrow\; \min_{a}.$$

В такой записи не осталось ничего от пошаговой истории, есть только суммарные потери и одна функция, которую надо подобрать, а с задачей такого вида уже можно работать, и она подскажет **направление**, в котором ансамбль надо двигать. Упрощение при этом именно временное. Направление найдётся только на обучающих объектах, поэтому к записи через b_k придётся вернуться.

Минимум и здесь ищется по функции, а напрямую такую задачу не решить, поэтому упрощение делается ещё раз. Забудем, что a — функция, и посмотрим на неё как на **вектор из N чисел** $\big(a(x_1), \dots, a(x_N)\big)$, ведь в написанной сумме участвуют только значения в обучающих точках, ничего другого от a там не требуется.

Перед нами оказывается обычная задача минимизации функции N переменных, а для такой есть стандартный инструмент: **градиентный спуск предписывает двигаться в сторону антиградиента** (negative

gradient). Посчитаем его в текущей точке, то есть при ансамбле a_{k-1}. Каждая координата градиента — производная функции потерь по предсказанию на своём объекте, и её принято обозначать буквой g:

$$g_i = \frac{\partial L(y_i, z)}{\partial z}\Big|_{z = a_{k-1}(x_i)}, \qquad s_i = -g_i.$$

- g_i — производная потерь по предсказанию на i-м объекте, вычисленная при текущем ансамбле;
- s_i — противоположная ей величина, координата антиградиента на том же объекте.

ОПРЕДЕЛЕНИЕ

Антиградиент (negative gradient) $s_i = -g_i$ — вектор из N чисел, который на каждом объекте обучающей выборки показывает, в какую сторону надо сдвинуть предсказание, чтобы суммарная ошибка убывала быстрее всего.

Сам приём знаком по неделям 3–4, где градиентным спуском подбирали веса линейной регрессии. Там производная бралась по вектору весов w, и шаг в сторону антиградиента двигал сами веса. Здесь механика та же, а различие в том, по чему берётся производная, — не по параметрам модели, а по её предсказаниям на обучающих объектах.

Координаты этого вектора называют **псевдоостатками** (pseudo-residuals): при квадратичной ошибке они совпадают с обычными остатками.

Оптимальную величину сдвига такой вектор не определяет, и лучше всего это видно на модуле отклонения $L(y, z) = |y - z|$. Антиградиент такой функции равен знаку разности, то есть $+1$ при $y > z$ и -1 при $y < z$, независимо от того, составляет промах копейку или миллион.

Длина шага из такого вектора не следует, и её выбирают отдельно.

Теперь подставим в общую конструкцию квадратичную ошибку, взяв её с множителем, который сократит двойку при дифференцировании:

$$L(y, z) = \frac{1}{2}(y - z)^2, \qquad g_i = -\big(y_i - a_{k-1}(x_i)\big), \qquad s_i = y_i - a_{k-1}(x_i) = r_i.$$

Антиградиент оказался буквально остатком при квадратичной функции. Значит, счёт по остаткам, проделанный вручную, не был отдельным эвристическим приёмом, а был шагом градиентного спуска, просто не названным своим именем.

Тем же дифференцированием антиградиент считается и для остальных функций потерь, а три самых употребительных случая удобно держать рядом.

функция потерь	$L(y, z)$	**антиградиент** s_i
квадратичная	$\frac{1}{2}(y - z)^2$	$y_i - a_{k-1}(x_i)$, то есть остаток
абсолютная	$\lvert y - z \rvert$	$\operatorname{sign}\big(y_i - a_{k-1}(x_i)\big)$
логистическая	$\ln(1 + e^z) - yz$	$y_i - p_i$

В строке про абсолютную ошибку производной в точке $y = z$ нет, и значение $\operatorname{sign}(0)$ там доопределяют нулём.

К логистической функции потерь лонгрид ещё вернётся в разделе 2.10, где разбирается бустинг для задачи классификации, и там же выводится третья строка таблицы.

INTERVIEW KIT

Остаток — частный случай антиградиента при квадратичной ошибке. Для любой другой дифференцируемой функции потерь остатки заменяются на $s_i = -g_i$, а остальная схема не меняется.

Направление такой вектор задаёт всегда, а оптимальную длину шага не определяет. Нагляднее всего это на модуле отклонения, где все координаты антиградиента равны ± 1 независимо от того, составляет промах копейку или миллион.

Стоит оговорить и происхождение всей конструкции. Идея последовательного ансамбля появилась раньше градиентного варианта, и первые работающие алгоритмы такого рода строились без всяких производных. Тем не менее выглядели они похоже: очередная модель обучалась не на исходной задаче, а на том, что осталось от предыдущих, и результат складывался, а не усреднялся. Различие пряталось в том, каким именно способом «то, что осталось» предъявлялось следующей модели.

ПРИМЕЧАНИЕ

Первым практичным алгоритмом такого рода стал **AdaBoost** (Freund, Schapire, 1995), который добивался похожего эффекта иначе. Вместо антиградиентов он после каждой итерации увеличивал веса неправильно классифицированных объектов, чтобы следующая модель уделила им больше внимания.

Позже выяснилось, что AdaBoost — частный случай градиентного бустинга с экспоненциальной функцией потерь. Сегодня в чистом виде он почти не используется, но именно с него началась линия последовательных ансамблей.

Градиентный вариант предложил Джером Фридман в 1999 году, и именно он заменил перевзвешивание объектов на движение по антиградиенту.

2.4. ПОЧЕМУ НЕЛЬЗЯ ПРОСТО ПРИБАВИТЬ АНТИГРАДИЕНТ

Направление спуска известно, и, казалось бы, остаётся прибавить его к текущему предсказанию на каждом объекте, после чего итерация закончена. Хочется, иначе говоря, вот такого шага:

$$a_k(x_i) \;=\; a_{k-1}(x_i) + \eta\, s_i, \qquad i = 1, \dots, N.$$

Так поступить не получится, и мешает не техническая трудность. Написанная строка определяет ансамбль только в N обучающих точках. Придёт новый объект x, и для него никакого s не считалось, а посчитать его не выйдет, потому что в производной g_i участвует истинный ответ y_i, которого на новом объекте нет. Прибавлять будет нечего.

Набор чисел — ещё не модель, ведь модель обязана выдавать предсказание в любой точке признакового пространства, а вектор s существует только на выборке.

Значит, вектор нужно превратить в функцию, которую можно вычислить где угодно. Вместо точного равенства $b_k(x_i) = s_i$ на обучающих объектах ищут приближённое,

$$b_k(x_i) \;\approx\; s_i, \qquad i = 1, \dots, N,$$

и шаг ансамбля возвращается к той записи, с которой всё начиналось, $a_k(x) = a_{k-1}(x) + \eta\, b_k(x)$, только теперь она определена всюду, а не в N обучающих точках.

Здесь и лежит цена всей конструкции. Исходная задача из раздела 2.1, точный arg min по функции потерь L, решена приближённо. Вместо неё ищется дерево, близкое к антиградиенту. Взамен задача вообще стала решаемой, и решается она стандартным обучением дерева.

Роль дерева при этом надо уточнить. Дерево выбрано ещё в разделе 2.1, но выбрано оно как **базис**, то есть как класс функций, из которого берутся поправки, а не как функция потерь и не как мера близости. Здесь добавляется ещё одно обстоятельство в его пользу, ведь дерево доопределяет поправку на всё признаковое пространство, тогда как вектор s живёт только на обучающей выборке. Новый объект попадёт в один из листьев и получит ту поправку, которая была нужна похожим на него обучающим объектам.

2.4.1. Чем мерить близость к антиградиенту

Здесь стоит остановиться, потому что задача незаметно сменилась. До сих пор речь шла об одной функции потерь L, по которой меряется качество ансамбля и из которой получен антиградиент. Теперь задача другая: приблизить деревом уже посчитанный вектор чисел $s_1, \dots, s_N$. Истинные ответы y_i в ней не участвуют, их место заняли требуемые поправки, а значит, и меру близости к ним можно выбирать независимо от L.

В классическом градиентном бустинге мерой близости служит квадратичная ошибка:

$$b_k = \arg\min_b \sum_{i=1}^{N} \big(b(x_i) - s_i\big)^2,$$

а это в точности та задача, которую решает обычное регрессионное дерево CART (classification and regression tree). Поэтому b_k обучают стандартным алгоритмом на выборке $\{(x_i,\ s_i)\}_{i=1}^N$, где вместо настоящих ответов стоят антиградиенты.

В классическом градиентном бустинге дерево подгоняется по MSE всегда, какой бы ни была L. Это две разные величины, и путать их не стоит.

величина	что делает	где работает
функция потерь $L(y, z)$	что минимизирует ансамбль, любая дифференцируемая	во всём алгоритме, из неё берётся антиградиент (таблица в разделе 2.3)
квадратичная ошибка	чем дерево приближает вектор s, всегда MSE	при обучении очередного дерева

У такого выбора есть внятный смысл. Антиградиент задаёт наилучшее направление в малой окрестности текущей точки, но выдать его целиком дерево не может: предсказания дерева постоянны внутри листа, поэтому доступны ему только кусочно-постоянные векторы (см. рисунок 17).

ПРИМЕЧАНИЕ

Откуда в названии слово «градиентный»

Минимизация суммы квадратов отклонений ищет **ближайшее к антиградиенту направление среди тех, которые дерево вообще способно выдать**, то есть проецирует направление спуска на класс деревьев. Поправки в бустинге, таким образом, задаёт градиент функции потерь — отсюда и название метода.

Приём здесь общий, и он уже встречался: когда точный критерий неудобен для поиска разбиения, его заменяют удобным, а найденные листья заполняют уже по исходному. На неделе 6 в этой роли

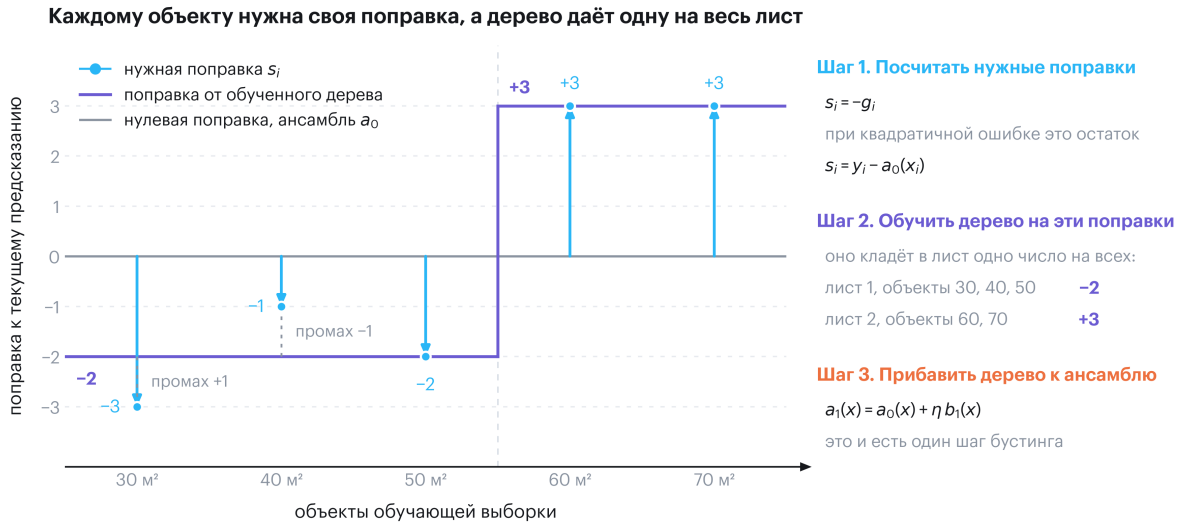


Рисунок 17. Антиградиент — произвольный вектор, а дерево способно выдать только кусочно-постоянную функцию

выступал индекс Джини, ведь дерево классификации растили по нему, хотя интересовала доля ошибок. Здесь на его месте стоит квадратичная ошибка на антиградиентах, и логика та же, ведь нужна она затем, чтобы дерево сумело выбрать разбиение, а настоящая L вернётся на шаге, где считаются числа в листьях.

ВАЖНО

Квадратичная ошибка здесь измеряет расстояние между деревом и вектором антиградиентов, а не качество предсказания ответов. Исходная L может быть какой угодно, в таблице раздела 2.3 это квадратичная, абсолютная или логистическая, и своё дело она уже сделала, когда породила s. Распространённое заблуждение — объяснять этот выбор словами «дерево иначе не умеет», ведь дерево обучается и по другим критериям. MSE здесь задаёт меру близости к антиградиенту; о возможностях дерева эта мера ничего не говорит.

Из той же формулировки видно, что происходит при обучении такого дерева. Регрессионное дерево при сплите по квадратичной ошибке ищет разбиение, после которого целевые значения внутри листа максимально однородны, а роль целевых значений сейчас играют требуемые поправки.

Обучение дерева по MSE на антиградиенты буквально означает просьбу собрать в один лист объекты с похожими нужными поправками, чтобы одно число в листе подошло им всем сразу.

Итог одной итерации умещается в три пункта.

1. Посчитать антиградиент $s_i = -g_i$ на каждом объекте по выбранной функции потерь L.
2. Обучить дерево по MSE на выборке $\{(x_i,\ s_i)\}$, получив разбиение на листья.
3. Положить в каждый лист число, посчитанное уже по настоящей L.

Первые два пункта разобраны, третий пока держится на умолчании, ведь в листе стоит среднее антиградиентов, как его и оставляет обучение дерева по MSE. Дальше выясняется, что это не лучший выбор.

Прежде чем заниматься третьим пунктом и устройством дерева, схему бустинга стоит записать целиком. Всё, что для этого нужно, уже разобрано.

2.5. АЛГОРИТМ ГРАДИЕНТНОГО БУСТИНГА ДЛЯ ЛЮБОЙ БАЗОВОЙ МОДЕЛИ И ПРОИЗВОЛЬНОЙ ФУНКЦИИ ПОТЕРЬ

Три пункта в конце предыдущего раздела записаны уже для дерева, а сама схема бустинга к дереву не привязана. Она складывается из результатов разделов 2.1–2.4 и о базовой модели не говорит ничего.

Устройство ансамбля пришло из раздела 2.1: поправки прибавляются по одной, каждая с темпом обучения η, а расхождение предсказания с ответом измеряется функцией потерь L. Там же поставлена и задача очередного шага.

Раздел 2.2 дал начальное приближение a_0, лучшую константу для выбранной функции потерь, с которой ансамбль начинает работу. В разделе 2.3 место остатков занял антиградиент s_i: при квадратичной ошибке он совпадает с остатком, а при любой другой дифференцируемой функции потерь остаётся определённым.

Последнее недостающее добавил раздел 2.4. Базовая модель приближает вектор антиградиентов, а близость к нему измеряется квадратичной ошибкой, и это отдельная величина, не связанная с выбором L.

Ниже эти результаты собраны в одну запись. Базовая модель обозначена в ней буквой b: на её месте может стоять решающее дерево, линейная модель или любая другая модель, способная приблизить заданный вектор чисел, и от этого выбора схема не меняется.

Постановка задачи. Градиентный бустинг

Дано. Обучающая выборка $\{(x_i, y_i)\}_{i=1}^{N}$ и функция потерь $L(y, z)$, дифференцируемая по второму аргументу.

Модель. Ансамбль набирается по одному слагаемому, начиная с начального приближения a_0:

$$a_k(x) = a_{k-1}(x) + \eta\, b_k(x),$$

где b_k — базовая модель произвольного вида, η — темп обучения.

Задача на шаге k. Найти базовую модель, которая сильнее всего уменьшит суммарные потери ансамбля:

$$b_k = \arg\min_{b} \sum_{i=1}^{N} L\big(y_i,\ a_{k-1}(x_i) + b(x_i)\big).$$

Напрямую эта задача не решается: минимум ищется по функции, и точной формулы у него нет. Приближённой заменой служит шаг 2 алгоритма из этого раздела, где точный arg min по L заменён приближением антиградиента.

Алгоритм градиентного бустинга

Шаг 0. Начальное приближение: $a_0 = \arg\min_c \sum_{i=1}^N L(y_i, c)$.

Шаги $k = 1, \dots, K$.

1. Посчитать антиградиент: $s_i = -\left.\frac{\partial L(y_i, z)}{\partial z}\right|_{z = a_{k-1}(x_i)}$, $i = 1, \dots, N$.
2. Приблизить его базовой моделью: $b_k = \arg\min_b \sum_{i=1}^N \big(b(x_i) - s_i\big)^2$.
3. Обновить ансамбль: $a_k(x) = a_{k-1}(x) + \eta\, b_k(x)$.

Ответ: $a_K(x)$.

2.6. ДЕРЕВО ИЗНУТРИ: СТРУКТУРА И ЗНАЧЕНИЯ ЛИСТЬЕВ

На место базовой модели дальше встаёт дерево, и в записи алгоритма оно остаётся чёрным ящиком: формулировка «приблизить вектор s базовой моделью» никак не использует того, что модель именно дерево, а не что угодно другое.

Между тем у дерева есть устройство, и из этого устройства можно извлечь выгоду.

Что дерево делает с объектом? Объект попадает в корень, проходит цепочку проверок вида «признак меньше порога» и в итоге оказывается ровно в одном листе. В листе лежит одно число, и это число дерево и выдаёт. В бустинге выданное число — поправка, прибавляемая к текущему предсказанию ансамбля.

Отсюда видно, что дерево собрано из двух частей, которые решают разные задачи. Первая — правила ветвления (splitting rules), которые определяют, как объекты раскладываются по листьям, и сами по себе не содержат никаких предсказаний. Вторая — набор чисел, по одному на лист.

Одну и ту же форму дерева можно наполнить какими угодно числами, и наоборот, по одним числам нельзя понять, кто в какой лист попал. Части при этом различимы, но не независимы: структура определяет состав листьев, а состав листа определяет наилучшее для него значение. Обеим частям нужны обозначения.

ОПРЕДЕЛЕНИЕ

Структура дерева k — разбиение признакового пространства на непересекающиеся области-листья $R_1, \dots, R_T$.

Значение в листе $w_j^{(k)}$ — число, лежащее в j-м листе k-го дерева, и именно оно становится поправкой для всех попавших туда объектов.

Область R_j можно читать двумя способами: как множество попавших в лист объектов и как кусок признакового пространства, который этот лист вырезает. Когда номер дерева ясен из контекста, верхний индекс опускается, и дерево записывается так:

$$b_k(x) \;=\; \sum_{j=1}^{T} w_j^{(k)}\, \mathbb{I}\big(x \in R_j\big).$$

- T — число листьев дерева;
- R_j — j-й лист, то есть область признакового пространства;
- $w_j^{(k)}$ — значение, лежащее в листе R_j у k-го дерева;
- $\mathbb{I}(x \in R_j)$ — индикатор попадания: единица, если объект оказался в листе R_j, и ноль иначе.

Запись выглядит громоздко, но описывает обычное поведение дерева. Листья не пересекаются, поэтому у конкретного объекта ровно один индикатор равен единице (тот, который отвечает его листу), а все прочие равны нулю.

Сумма из T слагаемых выбирает одно число w_j и обнуляет остальные, то есть выдаёт значение того листа, в который попал объект. На неделе 6 дерево вводилось в точности в таком виде, как разбиение пространства на непересекающиеся области плюс константа в каждой области.

ВАЖНО

Темп обучения η в эту запись не входит, он стоит снаружи дерева, в формуле обновления ансамбля $a_k(x) = a_{k-1}(x) + \eta\, b_k(x)$, а значения листьев записываются без него. Поэтому оптимальное w_j, которое отыскивается дальше, — это число, лежащее в листе, а не итоговая прибавка к предсказанию, ведь до ансамбля доходит $\eta\, w_j$.

Главное свойство этой записи в том, что дерево **кусочно-постоянно**. Оно зависит от x не гладко, а через разбиение признакового пространства на T непересекающихся кусков, и внутри каждого куска выдаёт одну и ту же константу всем объектам без исключения. Всё, что обученное дерево знает об объекте, — это номер его листа.

2.6.1. Что на самом деле делает сплит

Теперь, когда у обеих частей дерева есть имена, можно вернуться к вопросу, который в разделе 2.2 остался за кадром. Перебор порогов прошёл там механически: SSE посчитана для четырёх вариантов, наименьшая выбрана. За этой арифметикой стоит содержательный вопрос, и он же главный вопрос всего бустинга. Чем именно сплит $x \le 55$ лучше остальных и что вообще выигрывает дерево, когда ставит удачное разбиение?

Вернёмся в начало первой итерации, где в роли нужных поправок выступает уже известный столбец остатков:

$$-3, \quad -1, \quad -2, \quad +3, \quad +3.$$

Предположим сначала, что сплита нет и дерево состоит из одного листа. Тогда все пять квартир лежат в одной области R_1, а области полагается одно значение w_1 на всех. Лучшее общее значение при квадратичной ошибке — среднее:

$$w_1 = \frac{-3 - 1 - 2 + 3 + 3}{5} = 0.$$

Ноль. Дерево из одного листа не сдвинет ни одного предсказания, и после такого шага ансамбль останется тем же, чем был.

Причина не в том, что поправки малы: четыре из пяти по модулю не меньше двух, и работы более чем достаточно. Причина в том, что внутри листа объекты требуют поправок разного знака, ведь первым трём квартирам нужно вниз, последним двум вверх, и при усреднении они гасят друг друга.

Теперь поставим сплит $x \le 55$, который разводит те же пять чисел по двум листьям:

$$\underbrace{\{-3,\ -1,\ -2\}}_{R_1} \quad \text{и} \quad \underbrace{\{+3,\ +3\}}_{R_2}.$$

Противоречия внутри листьев больше нет, потому что слева всем нужно вниз, справа всем вверх, и каждому листу можно назначить осмысленное значение, $w_1 = -2$ и $w_2 = +3$. Это и есть числа, полученные перебором на первой итерации.

Насколько такой ход выгоден, видно по невязке, оставшейся необъяснённой. У одного листа со значением ноль она равна $9 + 1 + 4 + 9 + 9 = 32$, а у сплита $x \le 55$ левая поправка промахивается на трёх объектах на -1, $+1$ и 0, а правая не промахивается ни на одном, так что остаётся 2. Одно разбиение убрало тридцать единиц квадратичной ошибки из тридцати двух.

Полезно посмотреть и на проигравший вариант: сплит $x \le 65$ отделяет вправо единственную квартиру, а слева оставляет смесь из трёх отрицательных поправок и одной $+3$. Смесь снова гасит сама себя, её среднее равно $-0{,}75$, и такое значение не подходит ни одному из четырёх объектов, поэтому SSE остаётся $20{,}75$. Данные формально разделены, граница проведена, но листьям по-прежнему нечего назначить.

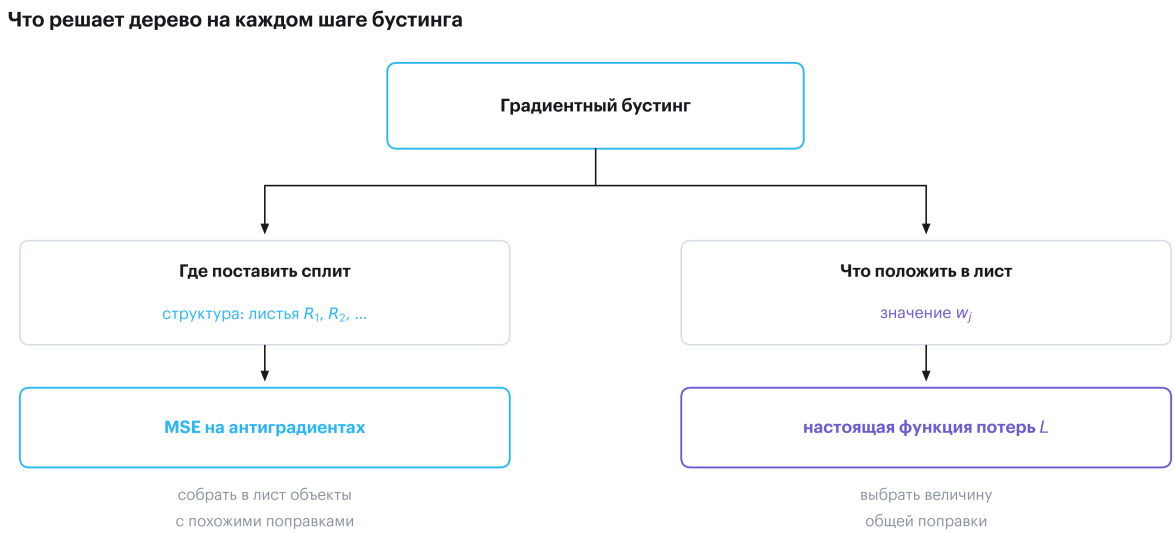


Рисунок 18. Структура и значения листьев решают одну общую задачу — построить хорошую поправку к текущей модели

Отсюда видно, за что отвечает каждая из двух частей дерева и по какому критерию каждая из них выбирается (см. рисунок 18).

ПРИМЕЧАНИЕ

Обучение дерева распадается на две задачи

Подбирать структуру и значения одновременно означало бы перебрать все возможные разбиения вместе со всеми наборами чисел в листьях, поэтому задачу делят надвое и решают по очереди.

Где резать. Здесь ищут сами области R_j, и это трудная часть, потому что точный перебор невозможен. Разбиение строят жадно, сверху вниз, и вдобавок по упрощённому критерию — квадратичной ошибке на антиградиентах.

Что положить в лист. Когда области зафиксированы и объекты по ним разложены, остаётся T независимых одномерных задач, по одной на каждый лист. Значения в листьях оптимизируют уже по исходной функции потерь L, и в общем случае этот критерий не совпадает с тем, по которому строилась структура.

Порядок именно такой: сначала определяют структуру, затем значения в листьях.

Вся свобода выбора сосредоточена в структуре: перебор порогов — это перебор способов сгруппировать объекты. Здесь функция потерь квадратичная, и потому оба критерия совпадают: значение листа — то же среднее поправок, которым уже считалась структура, $w_1 = -2$ и $w_2 = +3$. При другой функции потерь эти два числа разошлись бы. Отсюда и критерий качества сплита: чем однороднее поправки внутри каждой группы, тем он лучше. Порог $x \le 65$ показывает обратный случай: формально данные разделены, но слева осталась группа с большим разбросом.

Такой взгляд сразу ставит бустинговое дерево рядом с обычными.

дерево	сплит ищет группы, внутри которых
регрессии	похожи значения y
классификации	похожи метки классов
в бустинге	похожи **нужные поправки**

Разница, по сути, только в том, что стоит в целевом столбце: не сам ответ y_i, а поправка к тому, что ансамбль уже предсказал.

2.7. КАКОЕ ЧИСЛО ПОЛОЖИТЬ В ЛИСТ

Из кусочной постоянности следует одно техническое удобство, и оно во многом определяет весь дальнейший вывод. Запишем ошибку ансамбля после добавления k-го дерева, то есть сумму потерь по всем объектам обучающей выборки.

Каждый объект лежит ровно в одном листе, поэтому слагаемые можно пересобрать в другом порядке: сначала перебирать листья, а внутри листа его собственные объекты. Ничего при этом не теряется и не дублируется, каждое слагаемое учтено ровно один раз. Зато внутри листа $b_k(x_i)$ равно одному и тому же числу w_j:

$$\sum_{i=1}^{N} L\big(y_i,\ a_{k-1}(x_i) + b_k(x_i)\big) \ = \ \sum_{j=1}^{T} \sum_{i \in R_j} L\big(y_i,\ a_{k-1}(x_i) + w_j\big).$$

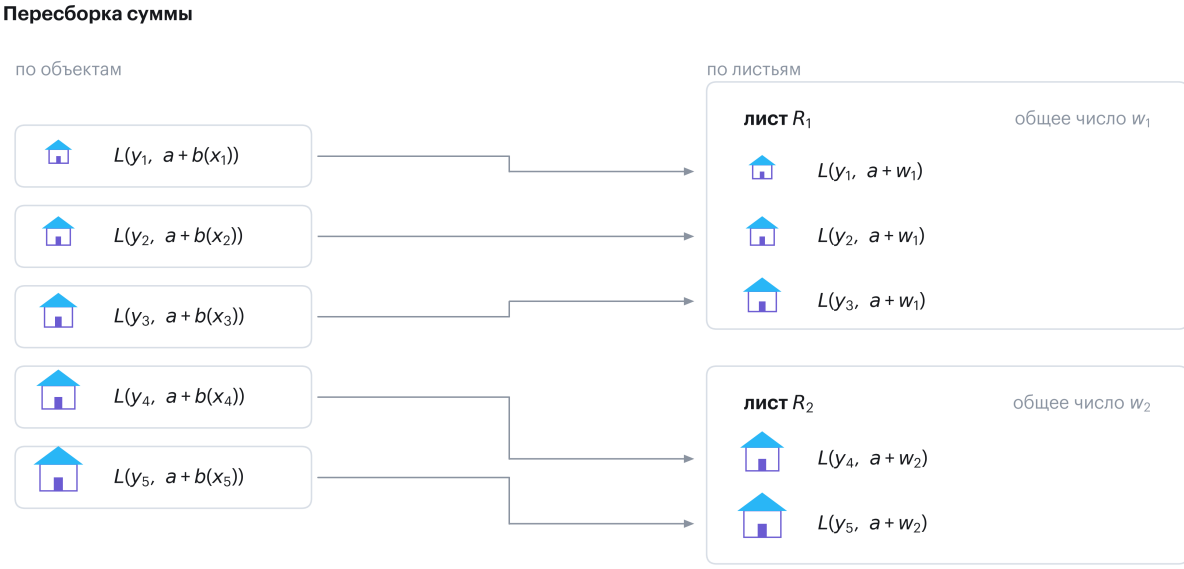


Рисунок 19. Каждый объект попадает ровно в один лист, поэтому слагаемые можно пересобрать по группам

Правая часть устроена так, что внешняя сумма состоит из T слагаемых, а слагаемое с номером j содержит только w_j, и ни w_1, ни w_{j+1}, ни любой другой вес в него не входит.

При таком разложении веса листьев не связаны между собой ничем, и изменение w_3 меняет вклад третьего листа, не затрагивая остальные $T-1$ слагаемых.

Значит, задача оптимизации по T переменным распадается на T независимых одномерных задач, по одной на каждый лист:

$$w_j^* = \arg\min_w \sum_{i \in R_j} L\big(y_i,\ a_{k-1}(x_i) + w\big).$$



Здесь стоит настоящая функция потерь, а не квадратичная.
Структура уже выбрана и не меняется, остаётся подобрать лучшее число для готовой группы.

Рисунок 20. Остаётся выбрать это число так, чтобы суммарная ошибка по листу была наименьшей

Здесь стоит **настоящая функция потерь** L, а не квадратичная. Приближение антиградиента по MSE понадобилось только затем, чтобы дерево вообще сумело выбрать разбиение.

Но разбиение к этому моменту уже выбрано, состав каждого листа известен и больше не меняется, так что остаётся простой вопрос: какое одно число лучше всего подходит готовой группе объектов, если качество мерить тем самым L, которое интересует на самом деле. Вместо целой структуры дерева здесь одна переменная, и минимизируется напрямую то, что нужно.

Стоит убедиться, что новая формула не противоречит ручному счёту. Для квадратичной ошибки $L(y, z) = \frac{1}{2}(y - z)^2$ сумма по непустому листу — парабола по w с положительным коэффициентом при квадрате, и её минимум достигается на среднем остатков листа:

$$w_j^* = \frac{1}{|R_j|} \sum_{i \in R_j} \big(y_i - a_{k-1}(x_i)\big).$$

Именно эти числа стояли в листьях в пункте 2.2, где левый лист первого дерева собрал остатки $-3,\ -1,\ -2$ и получил значение -2. При квадратичной ошибке дерево кладёт в листья оптимальные числа само, и пересчитывать после него нечего, а расхождение появляется тогда, когда L не квадратичная.

Для абсолютного отклонения $L(y, z) = |y - z|$ ответ получается другим, потому что сумма модулей минимизируется не средним, а **медианой** остатков листа. Дерево при этом строилось по квадратичной ошибке на знаках остатков, то есть структура и значения получены по двум разным правилам, и пересчёт листьев здесь обязателен.

INTERVIEW KIT

Структуру дерева и числа в листьях получают по разным правилам.

1. Структуру ищет обычный регрессионный CART по квадратичной ошибке на антиградиентах.
2. Значения листьев считают отдельно, уже по настоящей функции потерь.

Для квадратичной ошибки оба правила дают одно и то же число, среднее остатков, поэтому на регрессии разницы не видно, а для логлосса ответы расходятся примерно вчетверо. Частый уточняющий вопрос звучит так. Зачем тогда MSE, если минимизируется другое? Ответ в том, что MSE здесь измеряет близость дерева к вектору антиградиентов; качество самих предсказаний измеряет L, уже на шаге, где считаются значения листьев.

2.8. АЛГОРИТМ ГРАДИЕНТНОГО БУСТИНГА НАД ДЕРЕВЬЯМИ ДЛЯ ПРОИЗВОЛЬНОЙ ФУНКЦИИ ПОТЕРЬ

Общая схема из раздела 2.5 не менялась, изменилось только одно: базовой моделью b выбрано регрессионное дерево. Три шага из четырёх переходят в неё без изменений, а второй распадается надвое.

шаг общего алгоритма	во что он превращается для дерева
шаг 0, начальное приближение	без изменений
шаг 1, антиградиент	без изменений
шаг 2, приблизить антиградиент базовой моделью	распадается на две задачи: поиск структуры по квадратичной ошибке (раздел 2.6) и расчёт значений листьев по настоящей L (раздел 2.7)
шаг 3, обновление ансамбля	без изменений

Алгоритм

Градиентный бустинг над деревьями для произвольной функции потерь

Шаг 0. Начальное приближение $a_0 = \arg\min_c \sum_{i=1}^N L(y_i, c)$.

Шаги $k = 1, \dots, K$.

1. Посчитать антиградиенты $s_i = -\left.\dfrac{\partial L(y_i, z)}{\partial z}\right|_{z = a_{k-1}(x_i)}$ для всех $i = 1, \dots, N$.
2. Обучить **регрессионное дерево** по квадратичной ошибке на выборке $\{(x_i, s_i)\}_{i=1}^N$ и получить листья $R_1, \dots, R_T$.
3. Для каждого листа найти значение $w_j^* = \arg\min_w \sum_{i \in R_j} L\big(y_i, a_{k-1}(x_i) + w\big)$.
4. Обновить ансамбль: $a_k(x) = a_{k-1}(x) + \eta \sum_{j=1}^T w_j^* \mathbb{I}(x \in R_j)$.

Ответ: $a_K(x)$.

Шаги алгоритма закрывают две задачи из раздела 2.6:

- шаги 1 и 2 — первая задача, поиск структуры;
- шаг 3 — вторая задача, значения листьев.

Шаг 3 записан через arg min, а не формулой, и это не случайность. В общем случае минимум приходится искать численно. Как получить его формулой, разобрано в следующем разделе.

Те же шаги, выстроенные в конвейер, показаны на рисунке 21. Смотреть на нём стоит прежде всего на целевой столбец: он пересчитывается заново перед каждым деревом, и первое дерево не исключение — оно обучается на антиградиенте, взятом в точке a_0, а не на исходных ответах y_i.

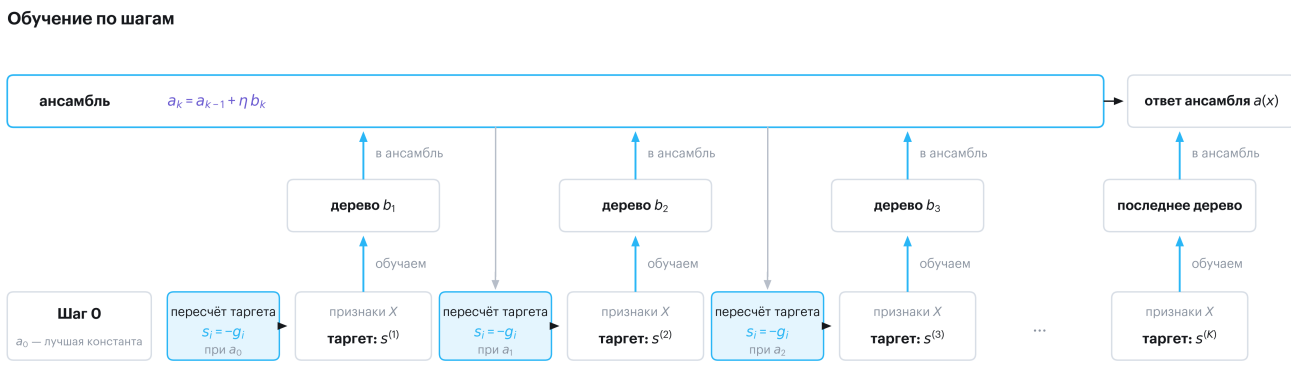


Рисунок 21. Цель пересчитывается перед каждым деревом, и первое не исключение: оно тоже обучается на антиградиенте

Полезно свести вместе, что уже сделано и что осталось.

шаг алгоритма	**регрессия, квадратичная ошибка**	**произвольная L**
начальное приближение a_0	среднее $\bar{y}$	лучшая константа, пункт 2.2.5
на чём учится дерево	остатки $y_i - a_{k-1}(x_i)$	антиградиент $s_i = -g_i$, раздел 2.3
чем меряется близость при поиске структуры	квадратичная ошибка	квадратичная ошибка, раздел 2.4
Что кладётся в лист	среднее остатков листа	$\arg\min_w \sum_{i \in R_j} L(\dots)$ — **решаем в следующем разделе**

Три пункта из четырёх, таким образом, закрыты: начальное приближение, антиградиент и поиск структуры разобраны в предыдущих разделах. Открытым остаётся четвёртый, где вместо формулы до сих пор стоит arg min, и его закрывает следующий раздел.

2.9. ЗНАЧЕНИЕ ЛИСТА: НЬЮТОНОВСКИЙ ШАГ

Общей схеме хватает первой производной: и антиградиент, и построение дерева, и шаг ансамбля обходятся ей. Вторая производная нужна там, где значение листа вычисляют ньютоновским шагом, как это сделано у Фридмана и в sklearn.

В листе стоит одномерная задача $\min_w \sum_{i \in R_j} L\big(y_i,\ a_{k-1}(x_i) + w\big)$, и для части функций потерь она решается точно.

функция потерь	**что кладётся в лист**	**нужна ли вторая производная**
квадратичная	среднее остатков, точный минимум	нет
модуль отклонения	медиана остатков, точный минимум	нет
логистическая	ньютоновский шаг, формула выводится в пункте 2.9.1	да

Когда замкнутого решения нет, остаётся численная минимизация для каждого листа каждого дерева, то есть десятки тысяч запусков за обучение. Вместо этого потери около текущей точки заменяют параболой и берут её минимум сразу. Вторая производная нужна именно для такой замены, а не из-за того, что функция потерь классификационная.

2.9.1. Заменить сложную функцию параболой

Приём стандартный и состоит в том, чтобы заменить неудобную функцию около текущей точки такой, минимум которой находится в одну строку, а простейшая функция с заведомо существующим минимумом — парабола.

Чтобы парабола вела себя рядом с нулём так же, как исходная функция, у них должны совпадать три величины: значение, наклон и кривизна. Наклон уже обозначен буквой g_i, а кривизна — вторая производная:

$$h_i = \left.\frac{\partial^2 L(y_i,\ z)}{\partial z^2}\right|_{z = a_{k-1}(x_i).}$$

Наклон и кривизна вместе определяют, где у параболы минимум: одного наклона для этого мало. Кривизна показывает, как быстро меняется наклон при движении, и в квадратичном приближении этого хватает, чтобы дойти до минимума за один шаг. При большой кривизне шаг короткий, при малой парабола пологая и шаг длинный.

Величину h_i в документации библиотек по традиции называют **гессианом**, хотя настоящий гессиан — матрица вторых производных, а здесь одно число. Название закрепилось настолько, что в библиотеках бустинга пользовательская функция потерь обязана вернуть два массива, `grad` и `hess`.

Требование совпадения трёх величин — это в точности разложение Тейлора до второго порядка, записанное по w около нуля для одного объекта:

$$L\big(y_i,\ a_{k-1}(x_i) + w\big) \ \approx\ \underbrace{L\big(y_i,\ a_{k-1}(x_i)\big)}_{\text{константа}} + g_i\, w \ +\ \frac{1}{2}\, h_i\, w^2.$$

Суммируем по объектам листа R_j, подставив общую для них поправку w. Первое слагаемое от w не зависит, ведь это уже накопленная ансамблем ошибка, и выбором веса её не изменить, поэтому для поиска минимума она бесполезна и отбрасывается.

$$\mathcal{L}_j(w) \ =\ w \sum_{i \in R_j} g_i \ +\ \frac{1}{2}\, w^2 \sum_{i \in R_j} h_i.$$

Обозначаем обе суммы по листу отдельными буквами.

$$\mathcal{L}_j(w) \ =\ G_j\, w + \frac{1}{2}\, H_j\, w^2, \qquad G_j = \sum_{i \in R_j} g_i, \qquad H_j = \sum_{i \in R_j} h_i.$$

- G_j — суммарный наклон ошибки по объектам листа R_j;
- H_j — суммарная кривизна ошибки по тем же объектам;
- $\mathcal{L}_j(w)$ — приближение потерь листа, обычная парабола от одного числа w.

Дифференцируем по w и приравниваем к нулю:

$$G_j + H_j\, w = 0 \qquad \Longrightarrow \qquad w_j \ =\ -\frac{G_j}{H_j}.$$

Полученная величина называется **ньютоновским шагом** (Newton step) в листе. Звёздочка у неё снята намеренно: w_j^* из раздела 2.7 обозначает точный минимум настоящей функции потерь, а w_j — минимум её квадратичного приближения. Для квадратичной ошибки эти две величины совпадают, потому что разложение Тейлора для параболы точное, а не приближённое.

Найденная точка — минимум. Для строго выпуклой функции потерь $h_i > 0$, поэтому сумма H_j тоже положительна, парабола направлена ветвями вверх, и её единственная стационарная точка лежит внизу.

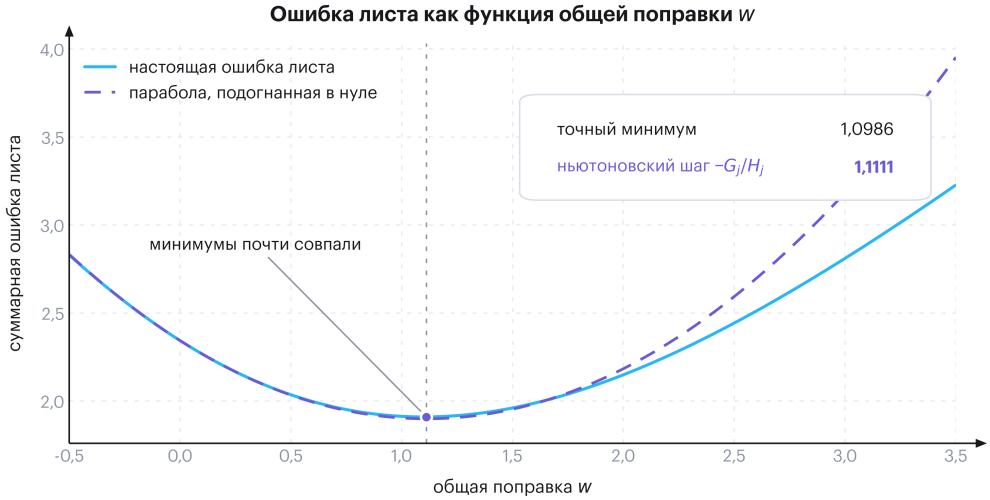


Рисунок 22. У параболы совпадают с исходной функцией значение, наклон и кривизна в текущей точке, а её минимум и есть ньютоновский шаг

INTERVIEW KIT

Ньютоновский шаг в листе

Дерево построено, объекты разложены по листьям. Для листа R_j считаются две суммы по его объектам:

$$G_j = \sum_{i \in R_j} g_i, \qquad H_j = \sum_{i \in R_j} h_i.$$

где g_i и h_i — первая и вторая производные функции потерь по предсказанию, взятые в точке текущего ответа ансамбля на объекте i.

В лист кладётся $w_j = -G_j / H_j$. Числитель суммирует наклоны по объектам листа, знаменатель — их кривизны: чем больше суммарная кривизна, тем короче шаг.

Метод Ньютона заменяет функцию параболой по значению, наклону и кривизне в текущей точке и берёт вершину этой параболы. Здесь та же замена сделана сразу для целой группы объектов, попавших в один лист, поэтому число получается формулой, а не численной минимизацией.

2.9.2. Ньютоновский шаг: числитель и знаменатель

Числитель G_j показывает, в какую сторону и насколько сильно объекты листа требуют сдвига. Наклоны разного знака при этом частично гасят друг друга, и суммарное требование получается слабым.

Знаменатель H_j показывает, насколько резко ошибка реагирует на сдвиг. При одном и том же G_j чем больше H_j, тем короче поправка, а считаются обе суммы по каждому листу отдельно, поэтому шаги в разных листьях выходят разной длины.

На квадратичной ошибке приближение ничего не портит: там все h_i равны единице, поэтому $H_j = |R_j|$, и отношение $-G_j / H_j$ обращается в среднее остатков листа.

ВАЖНО

Ньютоновский шаг не входит ни в антиградиент, ни в построение дерева, ни в шаг ансамбля. Дерево по-прежнему обучается регрессионным CART по квадратичной ошибке на антиградиенты, и критерий выбора сплитов ничего не знает о функции потерь L. Вторая производная входит ровно в одно число, то, которое кладут в лист.

Часть реализаций кладёт в лист среднее антиградиентов, то есть то же приближение с h_i, объявленными единицами. Кривизна в таком шаге не учтена, поэтому его длина не оптимальна, и промах компенсируют более мелким темпом обучения η и заметно большим K.

Гарантии убывания потерь на отдельной итерации нет и при ньютоновском шаге: направление приближено деревом, а шаг конечный, так что итерация может и увеличить потери.

Тем самым все четыре строки сводной таблицы закрыты: начальное приближение, антиградиент, структура дерева и значение листа считаются формулами. Требование к функции потерь при этом не выросло.

ПРИМЕЧАНИЕ

В классическом градиентном бустинге первая и вторая производные играют разные роли.

В классическом градиентном бустинге дерево строится как приближение антиградиента функции потерь. Поэтому при выборе сплитов используется информация первого порядка — значения первых производных.

После построения дерева для каждого листа выбирается величина шага. Если её вычисляют методом Ньютона, используется вторая производная.

Обрати внимание, в неделе 9 вторая производная работает шире. В XGBoost и LightGBM она входит и в критерий выбора разбиений, и в ограничения на минимальный лист, а в CatBoost способ вычисления значения листа выбирается параметром, и сплиты выбираются не по разнице лоссов.

2.10. ГРАДИЕНТНЫЙ БУСТИНГ ДЛЯ КЛАССИФИКАЦИИ

Про природу целевой переменной общая схема не предполагала ничего. Для бинарной классификации, $y_i \in \{0, 1\}$, алгоритм остаётся прежним, меняются только формулы, которые в него подставляются. Что именно меняется, видно построчно.

	регрессия, квадратичная ошибка	**классификация, логлосс**
что предсказывает ансамбль	сам ответ y	логит z, вероятность получается сигмоидой
функция потерь $L(y, z)$	$\frac{1}{2}(y - z)^2$	$\ln(1 + e^z) - y\,z$
градиент g_i	$a_{k-1}(x_i) - y_i$	$p_i - y_i$
кривизна h_i	1	$p_i(1 - p_i)$
начальное приближение a_0	$\bar{y}$	$\ln \frac{\bar{y}}{1-\bar{y}}$
значение листа $-G_j/H_j$	среднее остатков листа	$\frac{\sum_{i \in R_j}(y_i - p_i)}{\sum_{i \in R_j} p_i(1 - p_i)}$

Дальше строки этой таблицы разобраны по порядку.

Шкала предсказания. Сумма деревьев — произвольное вещественное число и вероятностью быть не может. Поэтому ансамбль предсказывает **логит** (logit), логарифм отношения шансов $z = \ln \frac{p}{1-p}$: он пробегает всю числовую прямую, складывать деревья ему ничто не мешает, а вероятность получается сигмоидой, $p = \sigma(z) = 1/(1 + e^{-z})$. Логит, сигмоида и логарифмические потери разобраны в неделе 5.

INTERVIEW KIT

В классификации бустинг не складывает вероятности, он складывает поправки к логиту. Сумма деревьев живёт на всей прямой, и сигмоида переводит её в интервал от нуля до единицы уже на выходе.

Производные. Если подставить сигмоиду в оба логарифма логлосса, вероятность из записи уходит, $L(y, z) = \ln(1 + e^z) - y\,z$, и дальше всё считается прямым дифференцированием по логиту:

- первая производная: $g_i = \sigma(z) - y_i = p_i - y_i$;
- вторая производная: $h_i = \sigma(z)\big(1 - \sigma(z)\big) = p_i\,(1 - p_i)$;
- лучшая константа: $a_0 = \ln \frac{\bar{y}}{1 - \bar{y}}$, то есть логит базовой частоты положительного класса.

Первый результат стоит отметить отдельно: **градиент логлосса по логиту равен ошибке предсказанной вероятности**. Дифференцирование шло по z, а выразилось всё через p. Антиградиент $-g_i = y_i - p_i$ выглядит как обычный остаток, но остатком «в шкале вероятностей» не является: по построению он измерен в тех единицах, в которых работает ансамбль, то есть в логитах.

Значение листа. Остаётся подставить найденные g_i и h_i в формулу $-G_j/H_j$ из раздела 2.9, и делается это в две строки.

Собираем суммы по листу из производных логлосса:

$$G_j = \sum_{i \in R_j} g_i = \sum_{i \in R_j} (p_i - y_i), \qquad H_j = \sum_{i \in R_j} h_i = \sum_{i \in R_j} p_i\,(1 - p_i).$$

Подставляем в $-G_j/H_j$, минус переносим в числитель:

$$w_j \;=\; -\frac{\sum_{i \in R_j} (p_i - y_i)}{\sum_{i \in R_j} p_i\,(1 - p_i)} \;=\; \frac{\sum_{i \in R_j} (y_i - p_i)}{\sum_{i \in R_j} p_i\,(1 - p_i)}.$$

Именно эту формулу приводят в учебниках и видео, обычно без вывода, и выглядит она загадочно. Отдельной идеи в ней нет, это всё то же $-G_j/H_j$ с подставленными производными логлосса, сумма антиградиентов в числителе и сумма кривизн в знаменателе. Встречается и вторая кодировка меток, ± 1, при которой ансамбль оценивает половину логарифма отношения шансов; коэффициенты в формулах от этого меняются, смысл остаётся прежним.

2.10.1. Считаем на числах

Возьмём те же пять квартир, но спросим не цену, а факт продажи за месяц: студия интересна узкому кругу покупателей, большая квартира дорога целиком и уходит долго, а средний сегмент разбирают быстро. Метки по порядку номеров получаются $0, 1, 1, 0, 0$, то есть зависимость **немонотонная**, единицы посередине и нули по краям.

Доля единиц равна $0{,}4$, поэтому $a_0 = \ln \frac{0{,}4}{0{,}6} = -0{,}4055$, и на старте всем объектам приписана одна и та же вероятность $p_i = 0{,}4$. Дальше считается всё, что нужно дереву:

№	x, м²	y_i	$g_i = p_i - y_i$	$h_i = p_i(1-p_i)$	антиградиент $-g_i$
1	30	0	$+0{,}4$	$0{,}24$	$-0{,}4$
2	40	1	$-0{,}6$	$0{,}24$	$+0{,}6$
3	50	1	$-0{,}6$	$0{,}24$	$+0{,}6$
4	60	0	$+0{,}4$	$0{,}24$	$-0{,}4$
5	70	0	$+0{,}4$	$0{,}24$	$-0{,}4$

Кривизна одинакова у всех только на первом шаге, пока одинаковы вероятности. Дерево обучается по квадратичной ошибке на столбец антиградиентов, и перебор тех же четырёх порогов снова приводит к сплиту $x \le 55$: справа оказываются два объекта с одинаковым антиградиентом $-0{,}4$, а любой другой порог перемешивает знаки сильнее. Слева остаются и $+0{,}6$, и $-0{,}4$, потому что одним пнём немонотонную тройку не разделить.

Левый лист: две оценки одной поправки. Объекты 1, 2 и 3 с метками $0, 1, 1$. Число в этот лист можно получить двумя способами, и они расходятся вчетверо:

- среднее антиградиентов, которое дерево кладёт туда само, обучаясь по квадратичной ошибке, $(-0{,}4 + 0{,}6 + 0{,}6)/3 = 0{,}2667$;
- ньютоновский шаг из раздела 2.9: суммы по листу равны $G_j = -0{,}8$ и $H_j = 0{,}72$, откуда $w_j = -G_j/H_j = 0{,}8/0{,}72 = 1{,}1111$.

Дело не в разных единицах измерения, ведь обе величины суть поправки к одному и тому же логиту. Оба числа отвечают на один вопрос, сколько прибавить объектам этого листа, и различаются тем, учтена ли кривизна. Среднее антиградиентов её не учитывает: деление на число объектов листа равносильно допущению «кривизна у всех единичная». Множитель $1/H_j$ ставит на её место настоящую, и здесь суммарная кривизна равна $0{,}72$ при трёх объектах, парабола пологая, а ньютоновский шаг оказывается в $3/0{,}72$ раза длиннее. Допущение $h_i = 1$ для квадратичной ошибки выполняется точно, а для логлосса $h_i = p_i(1-p_i) \le 0{,}25$, и здесь оно завышает кривизну в $4{,}17$ раза и во столько же укорачивает шаг.

У этой одномерной задачи есть и точный ответ, потому что на первом шаге все объекты листа имеют один и тот же логит. После прибавления w у них будет общая вероятность $\sigma(a_0 + w)$, а минимум логлосса достигается там, где она равна доле единиц в листе. Отсюда общая формула первого шага, $w_j = \ln \frac{n_1}{n_0} - a_{k-1}$, где n_1 и n_0 считают единицы и нули в листе, и в нашем случае

$$w = \ln \frac{2}{1} - \ln \frac{0{,}4}{0{,}6} = 1{,}0986.$$

Парабола промахнулась на $1{,}14\%$, а среднее антиградиентов промахнулось вчетверо (см. рисунок 23). Со второго шага такой сверки уже не будет: предсказания внутри листа разойдутся, общей вероятности $\sigma(a_{k-1} + w)$ не станет, а замкнутой формулы для точного минимума тоже. Искать его численно по-прежнему можно, но в каждом листе каждого дерева это слишком дорого, поэтому дальше используется ньютоновский шаг $-G_j/H_j$.

Правый лист: чистая группа. В нём объекты 4 и 5, обе метки нулевые, суммы дают $G_j = 0{,}8$ и $H_j = 0{,}48$, поэтому формула выдаёт $w_j = -1{,}6667$. Сверить это число не с чем: потери объекта с нулевой меткой

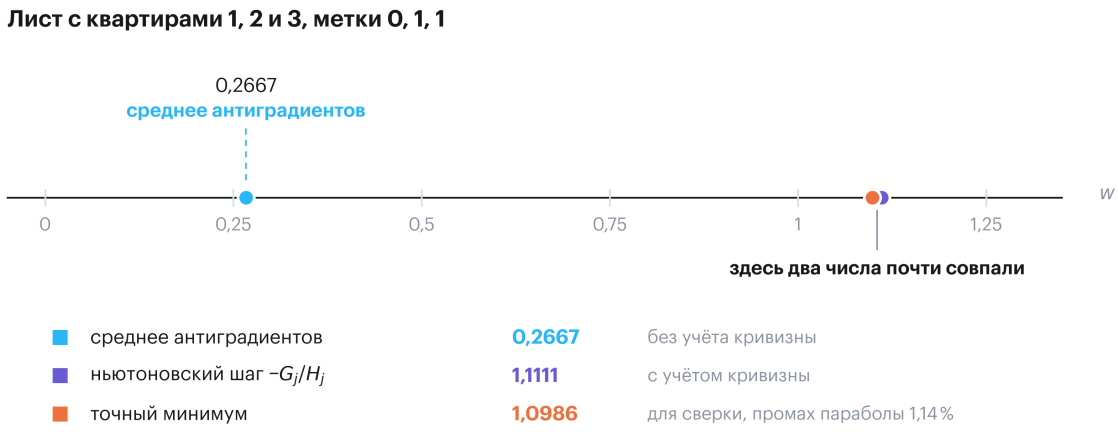


Рисунок 23. Среднее антиградиентов задаёт поправку, но не оптимальный шаг по настоящей функции потерь. Шаг Ньютона в этом примере заметно ближе к точному минимуму

равны $\ln(1 + e^z)$ и монотонно убывают по логиту, так что конечного решения у одномерной задачи нет и оптимум уходит на минус бесконечность.

Это не поломка формулы, а прямое следствие однородности листа: в нём нет объекта, который возражал бы против движения вниз. Конечное число парабола выдаёт лишь потому, что описывает потери около нуля и вдалеке от него перестаёт им соответствовать. Отсюда практический вывод: значения листьев приходится ограничивать. Отчасти это делает темп обучения, но он действует на все листья одинаково, а в неделе 9 в знаменателе появится добавка λ, которая ограничивает веса автоматически и тем сильнее, чем меньше объектов и кривизны собралось в листе.

Досчёт шага до конца. Остаётся применить $\eta = 0{,}5$ и посмотреть, куда сдвинулись вероятности:

лист	объекты	w_j	новый логит a_1	новая вероятность
левый	1, 2, 3	$+1{,}1111$	$-0{,}4055 + 0{,}5 \cdot 1{,}1111 = 0{,}1501$	$0{,}5375$
правый	4, 5	$-1{,}6667$	$-0{,}4055 - 0{,}5 \cdot 1{,}6667 = -1{,}2388$	$0{,}2246$

На самих потерях разница между двумя способами видна не хуже: у константной модели логлосс равен $0{,}673$, после одного шага он падает до $0{,}5043$ с параболическими весами и только до $0{,}6229$ со средними антиградиентов. Дерево, порог и объекты одни и те же, различаются лишь числа в листьях, и на этом различии теряется больше двух третей выигрыша.

2.10.2. Насыщение сигмоиды

Кривизна $h = p(1 - p)$ максимальна при $p = 0{,}5$ и стремится к нулю на обоих краях, поэтому в области насыщения (saturation) сдвиг логита почти не меняет вероятность. Градиент по краям ведёт себя несимметрично, потому что зависит ещё и от метки.

ситуация	$\lvert g_i \rvert$	h_i	вклад в лист
модель сомневается, p около $0{,}5$	около $0{,}5$	около $0{,}25$, максимум	и в числитель, и в знаменатель
уверена и права, p близко к y	близок к нулю	близка к нулю	почти ничего не даёт ни в числитель, ни в знаменатель
уверена и ошибается, p близко к $1 - y$	близок к единице	близка к нулю	много в числитель, почти ничего в знаменатель

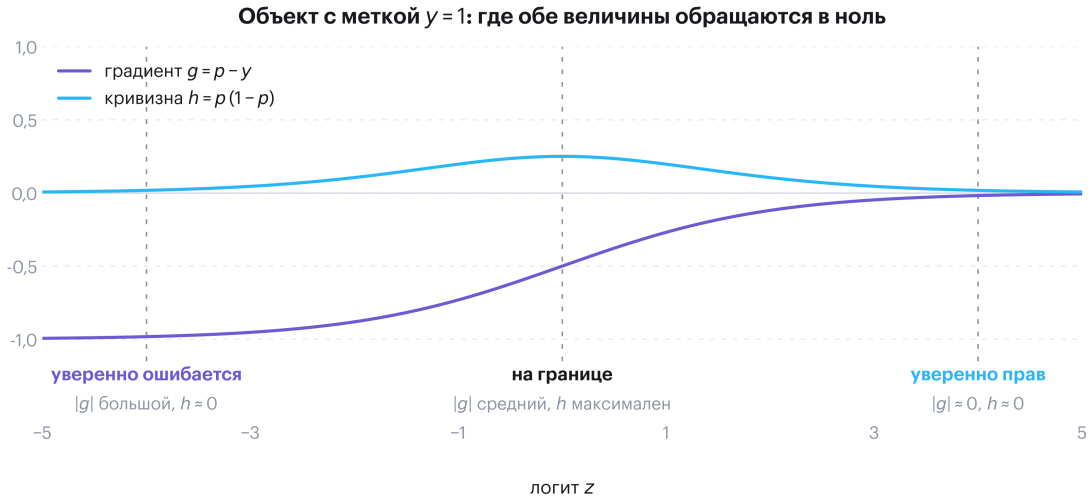


Рисунок 24. Градиент и кривизна обнуляются одновременно только там, где модель уверена и права. Уверенная ошибка, наоборот, тянет сильно

Числитель и знаменатель на таком объекте убывают одновременно, поэтому само отношение к нулю стремиться не обязано, и вклад объекта в значение листа определяется тем, как быстро убывает каждая из сумм.

Отсюда два вывода. **Вклад уверенно правильных объектов в обе суммы мал**, поэтому значения листьев формируются спорными объектами: бустинг сам концентрируется на границе между классами. **Уверенно ошибочный объект, наоборот, тянет вес листа на себя**, давая много в числитель и почти ничего в знаменатель, так что пара неверных меток в маленьком листе способна определить его значение.

2.11. БУСТИНГ В SKLEARN

Реализовывать бустинг руками не придётся, весь разобранный алгоритм в sklearn уже есть и называется `GradientBoostingRegressor` для регрессии и `GradientBoostingClassifier` для классификации.

Внутри у них ровно та схема, что в алгоритме бустинга над деревьями. На каждой итерации считается антиградиент, по нему обучается регрессионное дерево, значения листьев пересчитываются по выбранной функции потерь, и результат прибавляется к ансамблю с коэффициентом `learning_rate`.

Структура дерева при этом строится по квадратичной ошибке на антиградиентах, без участия вторых производных, а гессиан входит только в значение листа. В классификации это буквально один ньюто-

новский шаг: в коде `_gb.py` соответствующая ветка помечена комментарием `Make a single Newton-Raphson step` и считает $\sum(y_i - p_i) / \sum p_i(1-p_i)$ по объектам листа.

гиперпараметр	за что отвечает	типичное значение
`loss`	функция потерь L, из которой берётся антиградиент	`squared_error` в регрессии, `log_loss` в классификации
`learning_rate`	темп обучения η	0,1 и меньше
`n_estimators`	число деревьев K	подбирается под темп, а не берётся с запасом
`max_depth`	глубина базового дерева	от 3 до 6
`min_samples_leaf`	минимальный размер листа	1, поднимают на шумных данных
`subsample`	доля объектов на дерево	1,0, при меньшем получается стохастический бустинг
`max_features`	сколько признаков доступно на сплите	None, то есть все p
`n_iter_no_change`	сколько итераций без улучшения терпеть до остановки	None, то есть ранняя остановка выключена
`validation_fraction`	какая доля обучающей выборки уйдёт под эту проверку	0,1
`random_state`	фиксация подвыборок и отбора признаков	любое фиксированное число

В коде это выглядит так — и для классификации тоже, ведь у `GradientBoostingClassifier` набор параметров тот же.

```python
from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(
    loss='squared_error',     # 'absolute_error', 'huber', 'quantile'
    learning_rate=0.1,        # темп обучения η
    n_estimators=100,         # число деревьев K
    max_depth=3,              # глубина базового дерева, рабочий диапазон от 3 до 6
    min_samples_leaf=1,
    subsample=1.0,            # доля объектов на дерево, при <1 получается стохастический
    ↪ бустинг
    max_features=None,        # сколько признаков доступно на сплите
    n_iter_no_change=None,    # сколько итераций без улучшения терпеть до остановки
    validation_fraction=0.1, # какая доля обучающей выборки уйдёт на эту проверку
    random_state=0,
)
model.fit(X_train, y_train)
```

У классификатора вероятности выдаёт метод `predict_proba`, а функций потерь ему доступно две: `log_loss` из раздела 2.10 и `exponential`, при которой метод превращается в тот самый AdaBoost из исторической справки.

Ранняя остановка по умолчанию выключена, потому что `n_iter_no_change` равен None, и все `n_estimators` деревьев будут построены. Достаточно поставить туда число, и sklearn сам отложит долю `validation_fraction` от обучающей выборки под контроль качества.

2.11.1. Проверка на сквозном примере

Ручной счёт по сквозному примеру воспроизводится теми же тремя пнями и тем же темпом обучения, а результат можно сверить с таблицами по шагам.

```
from sklearn.metrics import mean_squared_error

X = [[30], [40], [50], [60], [70]]
y = [6, 8, 7, 12, 12]

model = GradientBoostingRegressor(learning_rate=0.5, n_estimators=3, max_depth=1).fit(X,
 ↪  y)

model.init_.constant_       # [[9.0]], начальное приближение a_0
model.predict(X)            # 6.75, 7.6875, 7.6875, 11.4375, 11.4375

tree = model.estimators_[0, 0].tree_
tree.threshold[0]           # 55.0, порог в корне первого дерева
tree.value[[1, 2], 0, 0]    # -2.0 и 3.0, значения двух его листьев

[mean_squared_error(y, p) for p in model.staged_predict(X)]   # 1.9, 0.775, 0.3531
```

Совпадает всё, включая падение ошибки по шагам и повтор порога 55 на второй итерации. Атрибут `init_` хранит начальное приближение отдельно от деревьев, `estimators_` содержит сами деревья обычными объектами `DecisionTreeRegressor`, а `staged_predict` выдаёт предсказания ансамбля после каждой добавленной модели.

Этой реализации хватает на задачи до нескольких десятков тысяч объектов, а дальше она упирается в стоимость жадного перебора всех порогов по всем признакам. Что с этим делают, разбирается в неделе 9, там же появится и второе семейство классов sklearn, написанное под большие данные.

2.12. ЧЕМ УПРАВЛЯЮТ В БУСТИНГЕ

Параметры перечислены, но у бустинга важнее не список, а порядок подбора, ведь лес прощает небрежную настройку, а бустинг нет. Дальше разобраны четыре параметра, которые действительно приходится настраивать, и связки между ними.

2.12.1. Темп обучения, `learning_rate`

Темп обучения η умножает вклад каждого дерева и не даёт ни одному из них закрыть ошибку целиком. Отсюда следствие: чем меньше темп, тем больше деревьев нужно для того же уровня качества. Зависимость нелинейная, так что правило «уменьшили темп вдвое, удвоили K» годится лишь как грубый ориентир.

За маленький темп платят временем обучения, и платить обычно стоит, потому что путь мелкими шагами устойчивее, а отдельная неудачная поправка распределяется по следующим. Эмпирическое правило такое: лучше работает стратегия, в которой η берут заведомо небольшим (меньше $0{,}1$), а число деревьев подбирают под него ранней остановкой.

2.12.2. Число деревьев и ранняя остановка, `n_estimators`, `n_iter_no_change`

В лесу добавить ещё сотню деревьев безопасно, а в бустинге рост K может ухудшить качество на тесте, потому что новые деревья переходят от структуры к шуму. У мелких деревьев кривая после минимума бывает почти плоской (см. рисунок 25).

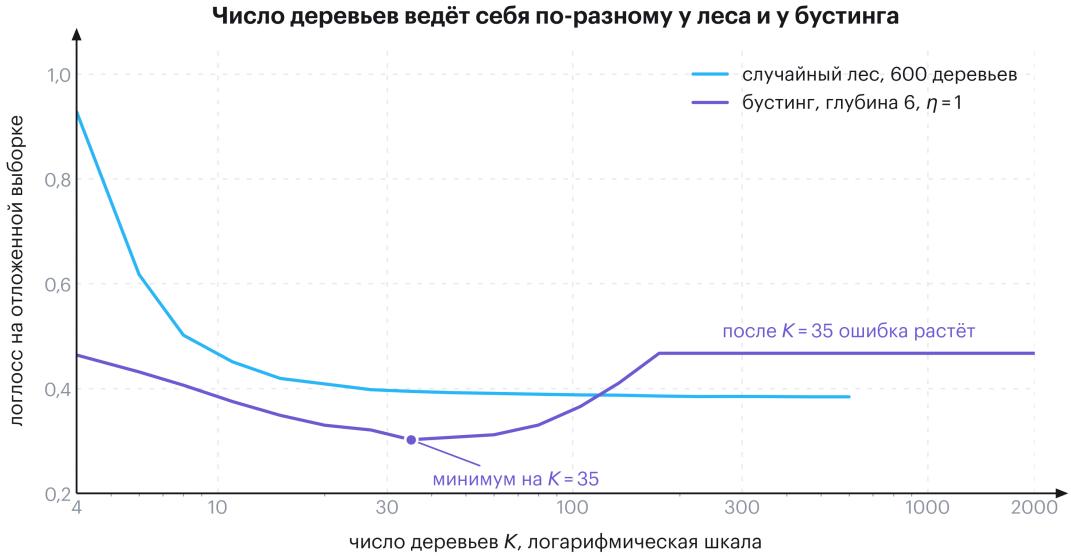


Рисунок 25. У леса кривая выходит на плато. У бустинга при $\eta = 1$ она заметно разворачивается вверх

Отсюда **ранняя остановка** (early stopping). Откладывается валидационная выборка, деревья добавляются по одному, качество на ней отслеживается, и если оно не улучшается заданное число итераций подряд, обучение прекращается. К лучшей итерации sklearn при этом автоматически не откатывается: в модели остаются все построенные деревья, включая добавленные после минимума. Если нужна именно лучшая итерация, её берут по `staged_predict`. Приём стандартный, но не обязательный, и при достаточном бюджете K подбирают обычным перебором.

Число деревьев в бустинге, таким образом, полноценный гиперпараметр: его число подбирают по валидации вместе с темпом обучения. Обратное преувеличение тоже вредно, ведь при малом темпе и мелких деревьях избыточное K часто обходится почти без потерь.

2.12.3. Глубина дерева, `max_depth`

Рабочий диапазон глубины — от 3 до 6, и у этих чисел есть содержательная причина. По пути от корня до листа дерево успевает проверить не больше признаков, чем его глубина, поэтому **глубина ограничивает порядок взаимодействий**, которые базовая модель способна выразить. Это полезная интуиция, а не точное равенство: один и тот же признак может встретиться на пути несколько раз. Речь при этом о шкале, в которой работает ансамбль, и для классификации это логит, а после сигмоиды картина сложнее.

Сумма пней складывается из вкладов отдельных признаков и взаимодействий не выражает вообще, а глубина 3 даёт взаимодействия не выше третьего порядка.

ПРИМЕР

Пусть цена квартиры зависит от площади и от этажа. Если оба признака действуют по отдельности (с ростом площади цена растёт, а на первом этаже падает), суммы пней достаточно: один пень отвечает за площадь, другой за этаж, и их вклады складываются.

Но если скидка за первый этаж возникает только в домах без лифта, отдельными вкладами такое не описать: нужен узел «этаж» под узлом «лифт», то есть глубина 2. Это и есть взаимодействие второго порядка.

В реальных задачах преобладают взаимодействия низких порядков, так что большая глубина редко окупается, зато повышает риск переобучения. Сложность дерева ограничивают и числом листьев T, и тогда диапазон другой: у мелких деревьев это единицы листьев. Два ограничителя одновременно обычно не задают, и основным здесь взят `max_depth`. Оба диапазона — эвристические ориентиры, а не следствие какого-либо расчёта.

2.12.4. Стохастический бустинг, `subsample`, `max_features`

Ничто не мешает обучать каждое дерево не на всей выборке, а на случайной её части, и выбирать сплит не среди всех p признаков, а среди случайных m. Это буквально идея Бреймана из части 1, перенесённая в последовательный ансамбль, и называется она **стохастическим градиентным бустингом** (stochastic gradient boosting). Типичная доля объектов — около половины, при большом N меньше. Эффектов три:

- **качество**: случайность мешает очередному дереву слишком точно подстроиться под остатки, и модель часто выходит точнее;
- **скорость**: дерево строится по доле данных, время падает примерно во столько же раз;
- **оценка**: у каждого дерева остаются объекты, не вошедшие в его подвыборку, и по ним считают оценку в духе OOB. Она менее надёжна, чем в лесу, поскольку эти объекты могли участвовать в обучении предыдущих деревьев, но как дешёвый ориентир работает.

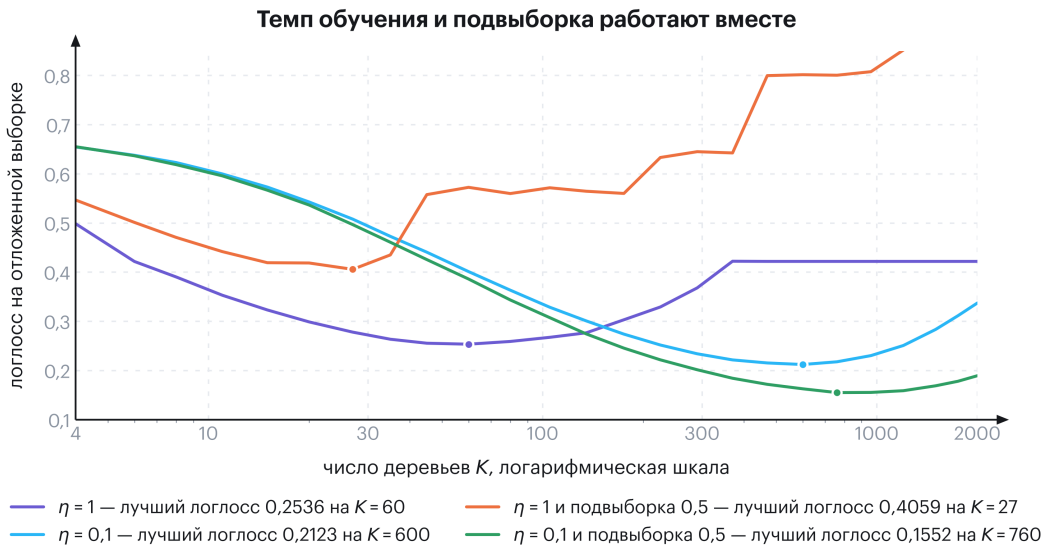


Рисунок 26. По отдельности подвыборка только мешает. В паре с малым темпом обучения она даёт лучший результат

Цель случайности при этом сместилась. В лесу она снижала корреляцию базовых моделей, чтобы усреднение давало больший выигрыш, а в бустинге служит ограничителем жадности, поскольку очередное дерево видит не все данные и потому не может подстроиться под остатки слишком точно.

Свой разброс есть и у бустинга, и удерживают его теми же средствами: малым темпом обучения, подвыборкой и ранней остановкой.

Сама по себе подвыборка на этой задаче проигрывает всем остальным режимам, а вместе с малым темпом обучения из пункта 2.12.1 даёт лучший результат из четырёх (см. рисунок 26). Темп здесь задан один раз на весь прогон: $\eta = 1$ означает, что поправка дерева прибавляется целиком, $\eta = 0{,}1$ — что каждый шаг укорочен в десять раз.

ПРИМЕЧАНИЕ

Порядок подбора, экономящий время:

1. Зафиксируй не слишком большое число деревьев, чтобы обучение шло быстро, и темп порядка $0{,}1$.
2. Под них подбери глубину в диапазоне от 3 до 6 и доли подвыборок по объектам и по признакам.
3. Для итоговой модели уменьши темп, увеличь число деревьев и определи его ранней остановкой, остальное оставив прежним.

Темп и число деревьев меняй вместе: подбирать одно при фиксированном другом обычно бессмысленно.

2.13. СРАВНЕНИЕ СЛУЧАЙНОГО ЛЕСА И ГРАДИЕНТНОГО БУСТИНГА

Оба метода строят много деревьев и объединяют их ответы, а расходятся в одной операции, из которой вытекает почти всё остальное.

$$\underbrace{a(x) = \frac{1}{K}\sum_{k=1}^{K} b_k(x)}_{\text{лес}} \qquad \underbrace{a_K(x) = a_0(x) + \eta\sum_{k=1}^{K} b_k(x)}_{\text{бустинг}}$$

Слева **среднее**: деревья равноправны, каждое решает исходную задачу целиком, а деление на K удерживает ответ в масштабе ответов базовой модели. Справа **сумма**: b_k решает не исходную задачу, а ту, что осталась после $k-1$ шагов, и новое слагаемое ансамбль не переусредняет, а сдвигает.

	Случайный лес	**Градиентный бустинг**
порядок построения	параллельно, деревья независимы	последовательно, каждое видит ошибки предыдущих
на что в первую очередь действует ансамблирование	на разброс, смещение остаётся смещением базового дерева	на смещение, за счёт последовательных поправок
какие деревья нужны	обычно глубокие и сильные	обычно неглубокие, от 3 до 6
рост K	качество выходит на плато	после оптимума качество обычно ухудшается, K подбирают
параллелизуемость обучения	полная	только внутри одного дерева
оценка без отложенной выборки	есть, OOB	обычно валидация или кросс-валидация; при `subsample` < 1 доступна OOB-диагностика

	Случайный лес	Градиентный бустинг
основные гиперпараметры	`n_estimators` и `max_features`	`learning_rate`, `n_estimators` и `max_depth`, подбираются вместе
чувствительность к настройке	низкая, значения по умолчанию обычно дают сильный baseline	высокая, разница между значениями по умолчанию и подобранными заметна
поведение на больших выборках	стоимость растёт с числом объектов и глубиной, обучение раскладывается по ядрам	классические реализации медленнее, для больших данных берут гистограммные

OOB-диагностика бустинга — не та же процедура, что в лесу: объект, не попавший в обучение текущего дерева, мог участвовать в обучении предыдущих, и оценка получается только по приросту качества на очередном шаге. В sklearn при `subsample` < 1 доступны `oob_improvement_`, `oob_scores_` и `oob_score_`.

Разницу в устройстве видно и по тому, какую границу каждый метод рисует на одних и тех же данных (см. рисунок 27).

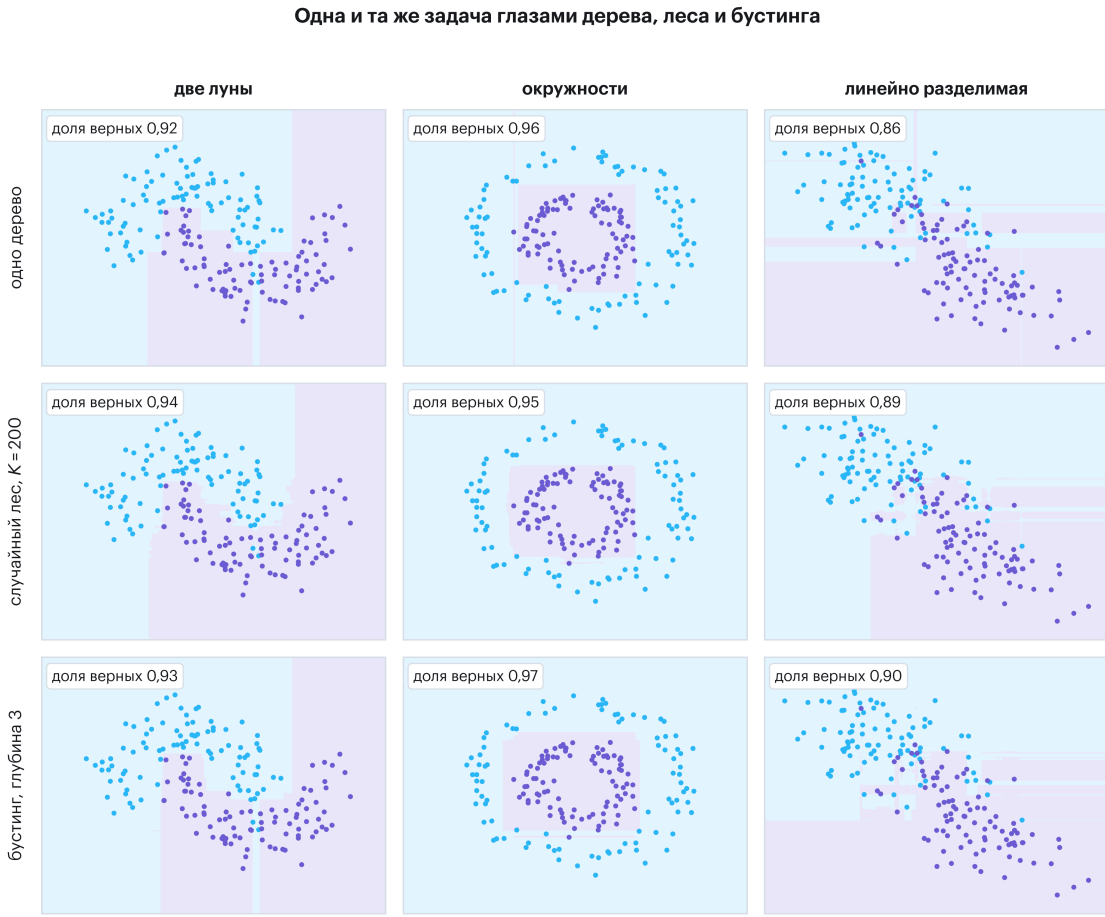


Рисунок 27. Одно дерево режет плоскость крупными прямоугольниками, лес усредняет сотни таких разбиений, бустинг набирает границу мелкими поправками

Одно дерево делит плоскость немногими прямоугольниками, и граница выходит грубо ступенчатой. Лес усредняет сотни таких разбиений, поэтому вокруг границы появляется размытая переходная зона, где ансамбль не уверен. Бустинг набирает границу мелкими поправками и рисует её резче леса, но аккуратнее одиночного дерева.

Доли верных ответов на отложенной выборке подписаны в углу каждой ячейки, и бустинг здесь нигде не выигрывает. Так и должно быть. На двумерных игрушечных задачах с сотней объектов настраивать нечего, а преимущество бустинг набирает на табличных данных с десятками признаков, где мелкие деревья успевают поймать нужные взаимодействия, а последовательные поправки работают со смещением, которого усреднение не трогает.

2.13.1. Лес и бустинг на пяти табличных задачах

Сравнение на словах стоит закончить числами. Пять наборов данных из sklearn, три конфигурации: лес с настройками по умолчанию, бустинг с настройками по умолчанию и бустинг после небольшого подбора. Взяты `RandomForestClassifier` и `RandomForestRegressor`, `GradientBoostingClassifier` и `GradientBoostingRegressor`, все с параметрами по умолчанию.

Подбор идёт по сетке из темпа обучения $0{,}05$ и $0{,}1$ и глубины 2, 3 и 4, число деревьев определяется ранней остановкой. Обучающая и тестовая части разделены в отношении три к одному при `random_state = 8`, конфигурация бустинга выбрана по отложенной части обучающей выборки, тестовая в подборе не участвовала.

набор данных	задача	объектов × признаков	метрика	лес, по умолчанию	бустинг, по умолчанию	бустинг после подбора
`breast_cancer`	классификация	569 × 30	доля верных	0,986	0,965	0,958
`wine`	классификация	178 × 13	доля верных	0,978	0,978	0,978
`digits`	классификация	1797 × 64	доля верных	0,982	0,964	0,976
`diabetes`	регрессия	442 × 10	R^2	0,343	0,335	0,390
`california_housing`	регрессия	20640 × 8	R^2	0,804	0,778	0,825

Время обучения на тех же прогонах:

набор данных	лес	бустинг по умолчанию	подбор бустинга
`breast_cancer`	0,1 с	0,2 с	1,2 с
`wine`	0,1 с	0,1 с	0,4 с
`digits`	0,1 с	3,5 с	23,9 с
`diabetes`	0,1 с	0,0 с	0,2 с
`california_housing`	0,7 с	1,9 с	44,5 с

Картина получается такая. Лес с настройками по умолчанию выигрывает или повторяет лучший результат на трёх задачах из пяти и нигде не проваливается, причём его параметры здесь не подбирались. Подобранный бустинг выигрывает на двух, и обе задачи регрессионные: на `diabetes` коэффициент детерминации растёт с $0{,}343$ до $0{,}390$, на `california_housing` — с $0{,}804$ до $0{,}825$.

Бустинг с настройками по умолчанию не выигрывает здесь нигде. Подбор улучшил его результат на `digits` и на двух регрессионных задачах, на `wine` результат не изменился, на `breast_cancer` ухудшился. Стоит он времени: на `california_housing` перебор по сетке занял $44{,}5$ секунды, тогда как одно обучение леса — $0{,}7$ секунды. Это сравнение двух разных процедур, а не двух обучений.

Наборы здесь небольшие, до двадцати тысяч объектов, и на них у леса преимущество: разброс он снимает усреднением сразу, а бустингу нужен подбор. На больших табличных данных с десятками признаков разрыв обычно смещается в сторону бустинга, и именно поэтому промышленные библиотеки из недели 9 построены на нём.

2.13.2. Почему в лесу деревья глубокие, а в бустинге мелкие

Разница в глубине выводится из той же операции. Лес уменьшает только разброс, поэтому вся работа со смещением ложится на отдельное дерево, и дерево растят без ограничений глубины. Глубокое дерево при этом не становится автоматически несмещённым, но оно хотя бы способно приблизить сложную зависимость, а пень такой возможности лишён.

В бустинге смещение уменьшается не внутри дерева, а между шагами, ведь каждое следующее обучается на том, что осталось от предыдущих, и точность набирается количеством слагаемых. Каждому отдельному b_k достаточно немного улучшить константное предсказание, а мелкое дерево здесь даже предпочтительнее.

Глубокое склонно за один шаг забрать почти весь сигнал вместе с шумом, и сумма тогда вырождается в одно большое дерево с мелкими поправками.

2.13.3. Что из части 1 переносится на бустинг, а что нет

Разложение ошибки на смещение, разброс и шум — тождество, верное для любой модели, включая бустинг. А вот конкретное выражение для дисперсии среднего, полученное в неделе 7, к бустингу неприменимо. При его выводе предполагалось, что базовые модели одинаково распределены и что все попарные корреляции между ними равны одному числу ρ.

В бустинге не выполняется ни то ни другое: b_k зависит от всех предыдущих деревьев, первое описывает крупную структуру, десятое уточняет детали, и корреляция между соседями не совпадает с корреляцией между первым и последним. Неприменима именно вероятностная модель, а не разложение.

Если кратко

Случайный лес

- **Лес — это бэггинг на деревьях плюс случайный выбор m признаков в каждом сплите.** Деревья обучаются независимо друг от друга, а ответом ансамбля становится их среднее.
- **Вторая случайность нужна не меньше первой.** Бутстрэп меняет состав объектов, а выбор m признаков в каждом узле мешает сильному признаку занять корень сразу у всех деревьев.
- **Усреднение снижает разброс, а смещение одинаково распределённых деревьев оставляет прежним.** Поэтому базовой моделью берут глубокое дерево, а предел задаёт $\rho\sigma^2$: качество упирается в корреляцию деревьев, а не в их число.
- **Главный рычаг в лесу — `max_features`.** Он сильнее прочих влияет на корреляцию деревьев, а с ней и на нижнюю границу ошибки. Число деревьев берут по бюджету, глубину не ограничивают.
- **OOB-оценка достаётся попутно.** Каждый объект предсказывают только те деревья, которые его не видели, поэтому для подбора гиперпараметров отдельная валидационная выборка не нужна.

Градиентный бустинг

- **Бустинг укладывается в одну цепочку.** Антиградиент функции потерь показывает, куда сдвинуть ответ на каждом объекте, объекты с похожими поправками собираются в листья дерева, и поправка входит в модель с коэффициентом η.
- **Дерево строят по одному правилу, а числа в листья кладут по другому.** Структура ищется по квадратичной ошибке на антиградиенты, а значение листа по настоящей L: для квадратичной ошибки это среднее остатков, для логистической ньютоновский шаг.
- **В классификации бустинг работает в шкале логитов**, а не вероятностей, ведь деревья складываются на логитах и только на выходе результат превращается в вероятность сигмоидой.
- **Рост числа деревьев в бустинге может ухудшить качество.** С ростом K ошибка на валидации может начать расти, поэтому число итераций часто выбирают ранней остановкой.
- **Лес и бустинг действуют на разные источники ошибки в первую очередь.** Усреднение независимых деревьев снижает разброс и почти не влияет на смещение, поэтому в лесу деревья делают глубокими. Последовательные поправки снижают смещение, поэтому в бустинге деревья делают мелкими. Это рабочая интуиция, а не строгое разделение: смещение и разброс меняются у обоих методов.
- **Записей алгоритма в части 2 три**, и различаются они по двум признакам: какой берётся базовая модель и какая функция потерь.

раздел	базовая модель	функция потерь	что добавляется
2.2.5	дерево	квадратичная	дерево обучается прямо на остатках
2.5	любая	произвольная	антиградиент вместо остатков
2.8	дерево	произвольная	структура по квадратичной ошибке, значения листьев по настоящей L

Запись из раздела 2.5 — общая, две другие получаются из неё подстановкой дерева на место базовой модели и квадратичной ошибки на место произвольной L.